

Unit 1

Chapter 1- Fundamentals of Object Oriented Programming

Introduction:

One characteristic that is constant in the software industry today is the "change". Change is one of the most critical aspects of software development and management. New tools and new approaches are announced almost every day. The impact of these developments is often very extensive and raises a number of issues that must be addressed by the software engineers. Most important among them are maintainability, reusability, portability, security, integrity, and user friendliness of software products.

To build today's complex software it just not enough to put together a sequence of programming statements and sets of procedures and modules. We need to use sound construction techniques and program structures that are easy to comprehend, implement and modify in a wide variety of situations.

Since the invention of the computer, many programming approaches have been tried. These include techniques such as modular programming, top-down programming, bottom-up programming and structured programming. The primary motivation in each case has been the concern to handle the increasing complexity of programs that are reliable and maintainable. These techniques became popular among programmers over the last two decades.

With the advent of languages such as C, structured programming became very popular and was the paradigm of the 1980s. Structured programming proved to be a powerful tool that enabled programmers to write moderately complex programs fairly easily. However, as the programs grew larger, even the structured approach failed to show the desired results in terms of bug-free, easy-to-maintain, and reusable programs.

Object-Oriented Programming (OOP) is an approach to program organization and development, which attempts to eliminate some of the pitfalls of conventional programming methods by incorporating the best of structured programming features with several new concepts. It is a new way of organizing and developing programs and has nothing to do with any particular language. However, not all languages are suitable to implement the OOP concepts easily. Languages that support OOP features include Smalltalk, Objective C, C++, Ada and Object Pascal. C++, an extension of C language, is the most popular OOP language today. C++ is basically a procedural language with object-oriented extension. Java, a pure object-oriented language, is one of the recent languages added to this list, the latest one being C#.

Object-Oriented Paradigm

The major objective of object-oriented approach is to eliminate some of the flaws encountered in the procedural approach. OOP treats data as critical element in the program development and does not allow it to flow freely around the system. It ties data more closely to the functions that operate on it and protects it from unintentional modification by other functions. OOP allows us to decompose a problem into a number of entities called Objects and then build data and functions (known as methods in Java) around these entities. The combination of data and methods make up an object (see Fig. 1.1).

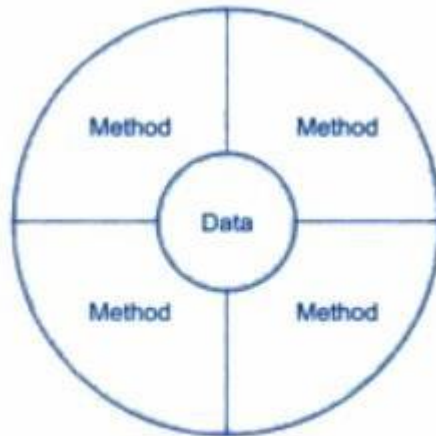


Fig. 1.1 *Object = Data + Methods*

The data of an object can be accessed only by the methods associated with that object. However, methods of one object can access the methods of other objects. Some of the features of object-oriented paradigm are:

- Emphasis is on data rather than procedure.
- Programs are divided into what are known as Objects.
- Data structures are designed such that they characterize the objects.
- Methods that operate on the data of an object are tied together in the data structure.
- Data is hidden and cannot be accessed by external functions
- Objects may communicate with each other through methods.
 - New data and methods can be easily added whenever necessary.
- Follows bottom-up approach in program design.

Object-oriented paradigm is the most recent concept among programming paradigms and still it means different things to different people. It is therefore important to have a working definition of object-oriented programming before we proceed further.

Our definition of object-oriented programming is: Object-oriented programming is an approach that provides a way of modularizing programs by creating partitioned memory area for both data and functions that can be used as templates for creating copies of such modules on demand. This means that an object is considered to be a partitioned area of computer memory that stores data and a set of operations that can access the data. Since the memory partitions are independent, the objects can be used in a variety of different programs without modifications.

Basic concepts of Object Oriented Programming

Objects and Classes

Objects are the basic runtime entities in an object-oriented system. They may represent a person, a place, a bank account, a table of data or any item that the program may handle. They may also represent user-defined data types such as vectors and lists. Any programming problem is analyzed in terms of objects and the nature of communication between them. Program objects should be chosen such that they match closely with the real-world objects. As pointed out earlier, an object takes up space in the memory and has an associated address like a record in Pascal, or a structure in C.

When a program is executed, the objects interact by sending messages to one another. For example, 'customer' and 'account' are two objects in a banking program, then the customer object may send a message to the account object requesting for the balance. Each object contains data and code to manipulate the data. Objects can interact without having to know the details of each other's data or code. It is sufficient to know the type of message accepted and the type of response returned by the objects. Although different authors represent them differently, Fig. 1.2 shows a notation that is popularly used to represent an object in object-oriented analysis and design.

We just mentioned that objects contain data and code to manipulate that data. The entire set of data and code of an object can be made a user-defined data type using the concept of a class. A class may be thought of as a 'data type' and an object as a 'variable' of that data type. Once a class has been defined, we can create any number of objects belonging to that class. Each object is associated with the data of type class with which they are created. A class is thus a collection of objects of similar type. For example, mango, apple and orange are members of the class fruit. Classes are user-defined data types and behave like the built-in types of a programming language. For example, the syntax used to create an object is no different than the syntax used to create an integer object in C. If fruit has been defined as a class, then the statement

```
fruit mango*
```

will create an object mango belonging to the class fruit.

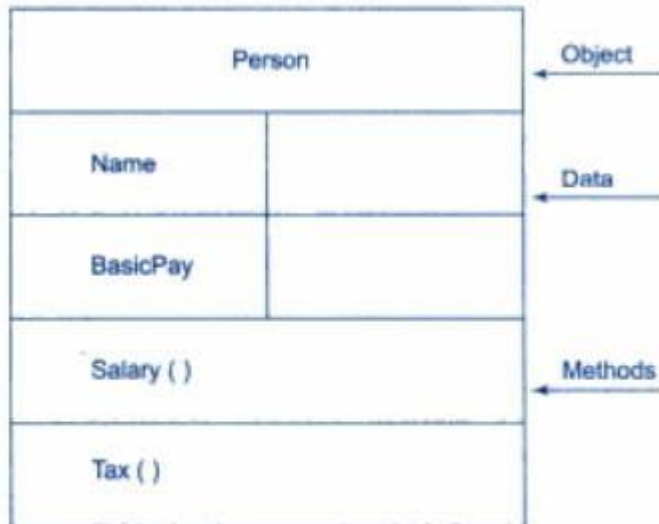


Fig. 1.2 Representation of an object

Data Abstraction and Encapsulation

The wrapping up of data and methods into a single unit (called class) is known as encapsulation. Data encapsulation is the most striking feature of a class. The data is not accessible to the outside world and only those methods, which are wrapped in the class, can access it. These methods provide the interface between the object's data and the program. This insulation of the data from direct access by the program is called data hiding. Encapsulation makes it possible for objects to be treated like "black boxes", each performing a specific task without any concern for internal implementation (see Fig. 1.3).



Fig. 1.3 Encapsulation—Objects as "black boxes"

Abstraction refers to the act of representing essential features without including the background details or explanations. Classes use the concept of abstraction and are defined as a list of abstract attributes such as size, weight and cost, and methods that operate on these attributes. They encapsulate all the essential properties of the objects that are to be created. Encapsulation is one of the three OOP principles, the other two being inheritance and polymorphism.

Inheritance

Inheritance is the process by which objects of one class acquire the properties of objects of another class. Inheritance supports the concept of hierarchical classification. For example, the bird robin is a part of the class flying bird, which is again a part of the class bird. As illustrated in Fig. 1.4, the principle behind this sort of division is that each derived class shares common characteristics with the class from which it is derived.

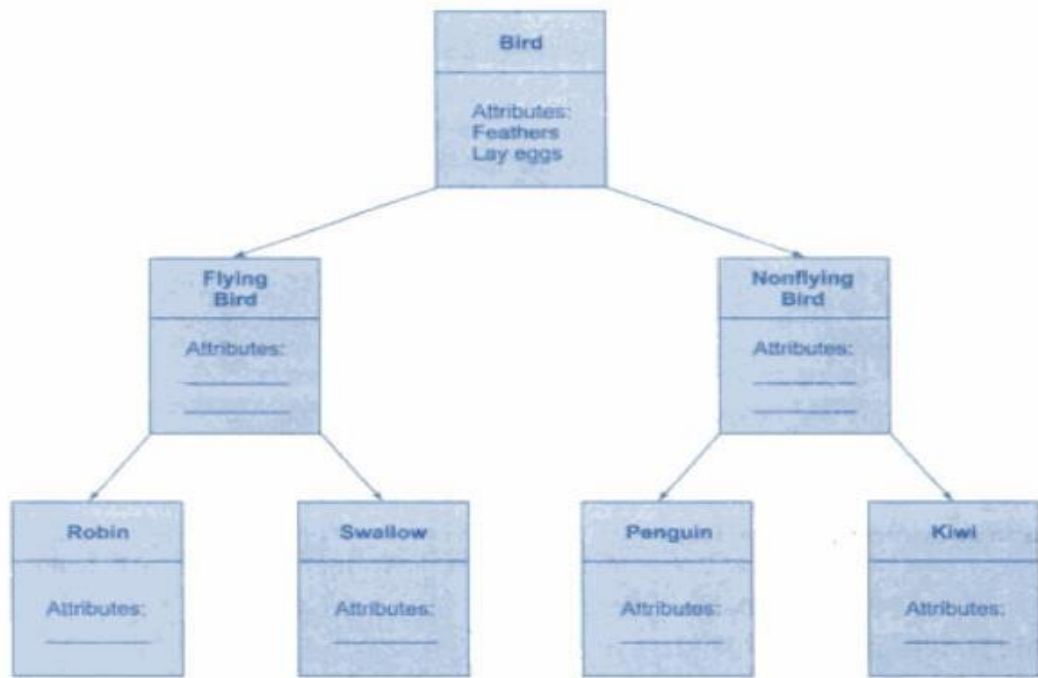


Fig. 1.4 Property inheritance

In OOP, the concept of inheritance provides the idea of reusability. This means that we can add additional features to an existing class without modifying it. This is possible by deriving a new class from the existing one. The new class will have the combined features of both the classes. Thus the real appeal and power of the inheritance mechanism is that it allows the programmer to reuse a class that is almost, but not exactly, what he wants, and to tailor the class in such a way that it does not introduce any undesirable side effects into the rest of the classes. In Java, the derived class is known as 'subclass'.

Note that each subclass defines only those features that are unique to it. Without the use of inheritance, each class would have to explicitly include all of its features.

Polymorphism

Polymorphism is another important OOP concept. Polymorphism means the ability to take more than one form. For example, an operation may exhibit different behaviour in different instances. The behaviour depends upon the types of data used in the operation. For example, consider the operation of addition. For two numbers, the operation will generate a sum. If the

operands are strings, then the operation would produce a third string by concatenation Figure 1.5 illustrates that a single function name can be used to handle different number and different types of arguments. This is something similar to a particular word having several different meanings depending on the context.

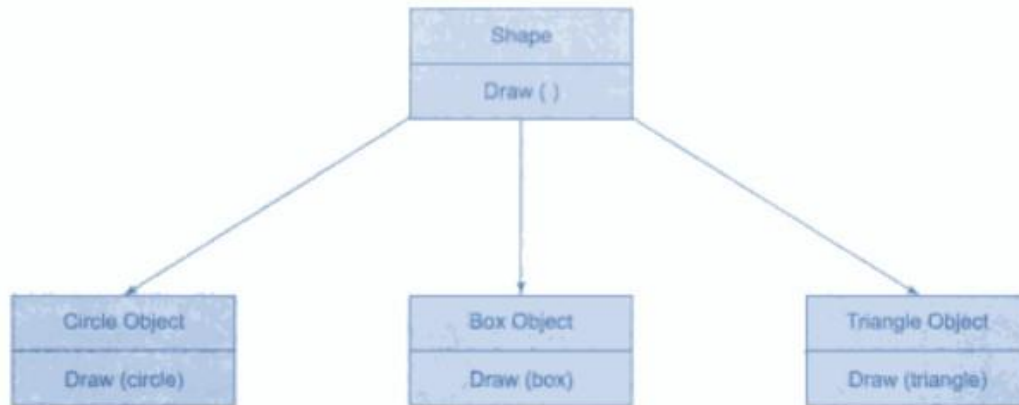


Fig. 1.5 Polymorphism

Polymorphism plays an important role in allowing objects having different internal structures to share the same external interface. This means that a general class of operations may be accessed in the same manner even though specific actions associated with each operation may differ. Polymorphism is extensively used in implementing inheritance.

Dynamic Binding

Binding refers to the linking of a procedure call to the code to be executed in response to the call. Dynamic binding means that the code associated with a given procedure call is not known until the time of the call at runtime. It is associated with polymorphism and inheritance. A procedure call associated with a polymorphic reference depends on the dynamic type of that reference.

Consider the procedure "draw" in Fig. 1.5. By inheritance, every object will have this procedure. Its algorithm is, however, unique to each object and so the draw procedure will be redefined in each class that defines the object. At run-time, the code matching the object under current reference will be called.

Message Communication

An object-oriented program consists of a set of objects that communicate with each other. The process of programming in an object-oriented language, therefore, involves the following basic steps:

1. Creating classes that define objects and their behaviour.
2. Creating objects from class definitions.
3. Establishing communication among objects.

Objects communicate with one another by sending and receiving information much the same way as people pass messages to one another as shown in Fig. 1.6. The concept of message passing makes it easier to talk about building systems that directly model or simulate their real-world counterparts.

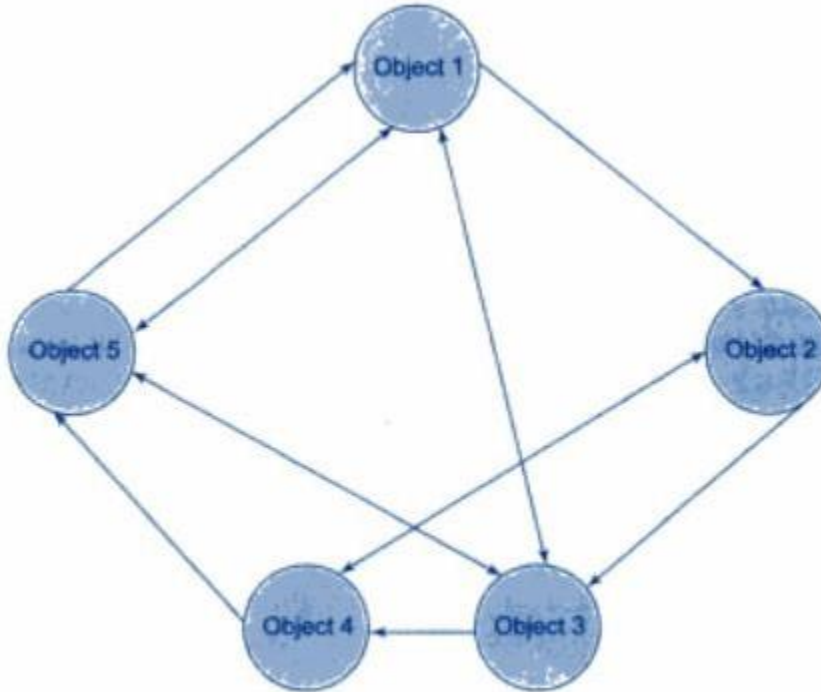


Fig. 1.6 Network of objects communicating between them

A message for an object is a request for execution of a procedure, and therefore will invoke a method (procedure) in the receiving object that generates the desired result, as shown in Fig. 1.7.

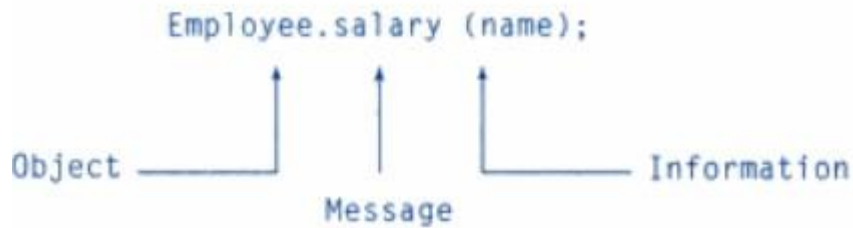


Fig. 1.7 Message triggers a method

Message passing involves specifying the name of the object, the name of the method (message) and the information to be sent. For example, consider the statement

Employee salary (name):

Here, Employee is the object, salary is the message and name is the parameter that contains information.



Objects have a life cycle. They can be created and destroyed. Communication with an object is feasible as long as it is alive.

Benefits of OOP

OOP offers several benefits to both the program designer and the user. Object-orientation contributes to the solution of many problems associated with the development and quality of software products. The new technology promises greater programmer productivity, better quality of software and lesser maintenance cost. The principal advantages are:

- Through inheritance, we can eliminate redundant code and extend the use of existing classes.
- We can build programs from the standard working modules that communicate with one another, rather than having to start writing the code from scratch. This leads to saving of development time and higher productivity.
- The principle of data hiding helps the programmer to build secure programs that cannot be invaded by code in other parts of the program.
- It is possible to have multiple objects to coexist without any interference. . It is possible to map objects in the problem domain to those objects in the program.
- It is easy to partition the work in a project based on objects.
- The data-centered design approach enables us to capture more details of a model in a implementable form.
 - Object-oriented systems can be easily upgraded from small to large systems.
 - Message passing techniques for communication between objects make the interface descriptions with external systems much simpler.
 - Software complexity can be easily managed.

While it is possible to incorporate all these features in an object-oriented system, their importance depends on the type of the project and the preference of the programmer. There are a number of issues that need to be tackled to reap some of the benefits stated above. For instance, class libraries must be available for reuse. The technology is still developing and current products may be superseded quickly. Strict controls and protocols need to be developed if reuse is not to be compromised.

A software that is easy to use is hard to build. It is hoped that the object-oriented programming languages like C++ and Java would help manage this problem.

Applications of OOP

OOP is one of the programming buzzwords today. There appears to be a great deal of excitement and interest among software engineers in using OOP. Applications of OOP are beginning to gain importance in many areas. The most popular application of object-oriented programming, up to now, has been in the area of user interface design such as windows. There are hundreds of windowing systems developed using OOP techniques.

Real-business systems are often much more complex and contain many more objects with complicated attributes and methods. OOP is useful in this type of applications because it can simplify a complex problem. The promising areas for application of OOP include:

- Real-time systems
- Simulation and modelling
- Object-oriented databases
- Hypertext, hypermedia and expert text
- AI and expert system.
- Neural networks and parallel programming
- Decision support and office automation systems
- CIM/CAD/CAD system

It is believed that the richness of OOP environment will enable the software industry to improve not only the quality of software systems but also its productivity. Object-oriented technology is certainly changing the way software engineers think, analyze, design and implement systems today.

Chapter 2 Introduction to Java

Java Features

The inventors of Java wanted to design a language which could offer solutions to some of the problems encountered in modern programming. They wanted the language to be not only reliable, portable and distributed but also simple, compact and interactive. Sun Microsystems officially describes Java with the following attributes:

<i>Java 2 Features</i>	<i>Additional Features of J2SE 5.0</i>
• Compiled and Interpreted • Platform-Independent and Portable • Object-Oriented • Robust and Secure • Distributed • Familiar, Simple and Small • Multithreaded and Interactive • High Performance • Dynamic and Extensible	• Ease of Development • Scalability and Performance • Monitoring and Manageability • Desktop Client • Core XML Support • Supplementary character support • JDBC RowSet

Although the above appears to be a list of buzzwords, they aptly describe the full potential of the language. These features have made Java the first application language of the World Wide Web. Java will also become the premier language for general purpose stand-alone applications.

Compiled and Interpreted

Usually a computer language is either compiled or interpreted. Java combines both these approaches thus making Java a two-stage system. First, Java compiler translates source code into what is known as bytecode instructions. Bytecodes are not machine instructions and therefore, in the second stage, Java interpreter generates machine code that can be directly executed by the machine that is running the Java program. We can thus say that Java is both a compiled and an interpreted language.

Platform-Independent and Portable

The most significant contribution of Java over other languages is its portability. Java programs can be easily moved from one computer system to another, anywhere and anytime. Changes and upgrades in operating systems, processors and system resources will not force any changes in Java programs. This is the reason why Java has become a popular language for programming on Internet which interconnects different kinds of systems worldwide. We can download a Java applet from a remote computer onto our local system via Internet and execute it locally. This makes the Internet an extension of the user's basic system providing practically unlimited number of accessible applets and applications.

Java ensures portability in two ways. First, Java compiler generates bytecode instructions that can be implemented on any machine. Secondly, the size of the primitive data types are machine independent.

Object-Oriented

Java is a true object-oriented language. Almost everything in Java is an object. All program code and data reside within objects and classes. Java comes with an extensive set of classes, arranged in packages, that we can use in our programs by inheritance. The object model in Java is simple and easy to extend.

Robust and Secure

Java is a robust language. It provides many safeguards to ensure reliable code. It has strict compile time and run time checking for data types. It is designed as a garbage-collected language relieving the programmers virtually all memory management problems. Java also incorporates the concept of exception handling which captures series errors and eliminates any risk of crashing the system.

Security becomes an important issue for a language that is used for programming on Internet. Threat of viruses and abuse of resources are everywhere. Java systems not only verify all memory access but also ensure that no viruses are communicated with an applet. The absence of pointers in Java ensures that programs cannot gain access to memory locations without proper authorization.

Distributed

Java is designed as a distributed language for creating applications on networks. It has the ability to share both data and programs. Java applications can open and access remote objects on Internet as easily as they can do in a local system. This enables multiple programmers at multiple remote locations to collaborate and work together on a single project.

Simple, Small and Familiar

Java is a small and simple language. Many features of C and C++ that are either redundant or sources of unreliable code are not part of Java. For example, Java does not use pointers, pre processor header files, go to statement and many others. It also eliminates operator overloading and multiple inheritance. For more detailed comparison of Java with C and C++, refer to Section 2.3.

Familiarity is another striking feature of Java. To make the language look familiar to the existing programmers, it was modelled on C and C++ languages. Java uses many constructs of C and C++ and therefore, Java code "looks like a C++" code. In fact, Java is a simplified version of C++.

Multithreaded and Interactive

Multithreaded means handling multiple tasks simultaneously. Java supports multithreaded programs. This means that we need not wait for the application to finish one task before beginning another. For example, we can listen to an audio clip while scrolling a page and at the same time download an applet from a distant computer. This feature greatly improves the interactive performance of graphical applications.

The Java runtime comes with tools that support multi process synchronization and construct smoothly running interactive systems,

High Performance

Java performance is impressive for an interpreted language, mainly due to the use of intermediate byte code. According to Sun, Java speed is comparable to the native C/C++. Java architecture is also designed to reduce overheads during runtime. Further, the incorporation of multi reading enhances the overall execution speed of Java programs.

Dynamic and Extensible

Java is a dynamic language. Java is capable of dynamically linking in new class libraries, methods, and objects. Java can also determine the type of class through a query, making it possible to either dynamically link or abort the program, depending on the response.

Java programs support functions written in other languages such as C and C++. These functions are known as native methods. This facility enables the programmers to use the efficient functions available in these languages. Native methods are linked dynamically at runtime.

Ease of Development

Java 2 Standard Edition (J2SE) 5.0 supports features, such as Generics, Enhanced for Loop, Autoboxing or unboxing, Typesafe Enums, Varargs, Static import and Annotation. These features reduce the work of the programmer by shifting the responsibility of creating the reusable code to the compiler. The resulting source code is free from bugs because the errors made by the compiler are less when compared to those made by programmers. Thus, each of the linguistic features is designed to develop Java programs in an easier way.

Scalability and Performance

J2SE 5.0 assures a significant increase in scalability and performance by improving the start up time and reducing the amount of memory used in Java 2 runtime environment. For example, the introduction of the class, data sharing in the Hotspot Java Virtual Machine (JVM) improves the start up time by loading the core classes from the jar files into a shared archive. Memory utilization is reduced by sharing data in the shared archive among multiple JVM processes. In the earlier versions, the data was replicated in each JVM instance.

Monitoring and Manageability

Java supports a number of APIs, such as JVM Monitoring and Management API, Sun Management Platform Extension, Logging, Monitoring and Management Interface, and Java Management Extension (JMX) to monitor and manage Java applications. For example, Java provides JVM Monitoring and Management API to track the information at the application level and JVM level when deploying a large application. Java provides tools, such as jconsole, jps, jstat, and jstatd to make use of monitoring and management facilities. For example, GUI based tool called jconsole is used to monitor the JVM.

Desktop Client

J2SE 5.0 provides enhanced features to meet the requirements and challenges of the Java desktop users. It provides an improved Swing look and feel called Ocean. This feature is mainly used for developing graphics applications that require OpenGL hardware acceleration.

Miscellaneous Features

In addition to the above features, J2SE 5.0 supports the features such as:

Core XML Support J2SE 5.0 adds a powerful XML feature to the Java platform. Java contains some special packages for interface, to instantiate Simple API for XML (SAX) and Document Object Model (DOM) parsers to parse an XML document, transform the content of an XML document, and validate an XML document against the schema.

Supplementary Character Support Java adds the 32-bit supplementary character support as part of the Unicode 4.0 support. The supplementary characters are encoded with UTF-16 values to generate a different character called, surrogate codepoint.

JDBC RowSet

Java supports JDBC RowSet to send data in a tabular format between the remote components of a distributed enterprise application. JDBC RowSet contains CachedRowSet and WebRowSet objects. The CachedRowSet object is a JavaBean component which acts like a container. This object contains a number of rows of data, which are retrieved from the database. The data stored in the CachedRowSet can be directly accessed without connecting to the database or any other data source. The rows of data that are retrieved from the database can be synchronized later. The WebRowSet object can operate without being connected to the database or data source. The WebRowSet object uses XML format to read and write the rowset.

Java Environment

Java environment includes a large number of development tools and hundreds of classes and methods. The development tools are part of the system known as Java Development Kit (JDK) and the classes and methods are part of the Java Standard Library (JSL), also known as the Application Programming Interface (API).

Java Development Kit

The Java Development Kit comes with a collection of tools that are used for developing and running Java programs. They include:

- appletviewer (for viewing Java applets)
- javac (Java compiler)
- java (Java interpreter)
- javap (Java disassembler)
- javah (for C header files)
- javadoc (for creating HTML documents)
- jdb (Java debugger)

Table 2.3 lists these tools and their descriptions.

Table 2.3 Java Development Tools	
Tool	Description
appletviewer	Enables us to run Java applets (without actually using a Java-compatible browser).
java	Java interpreter, which runs applets and applications by reading and interpreting bytecode files.
javac	The Java compiler, which translates Java sourcecode to bytecode files that the interpreter can understand.
javadoc	Creates HTML-format documentation from Java source code files.
jvah	Produces header files for use with native methods.
javap	Java disassembler, which enables us to convert bytecode files into a program description.
jdb	Java debugger, which helps us to find errors in our programs.

The way these tools are applied to build and run application programs is illustrated in Fig. 2.5. To create a Java program, we need to create a source code file using a text editor. The source code is then compiled using the Java compiler javac and executed using the Java interpreter java. The Java debugger jdb is used to find errors, if any, in the source code. A compiled Java program can be converted into a source code with the help of Java disassembler javap. We learn more about these tools as we work through the book.

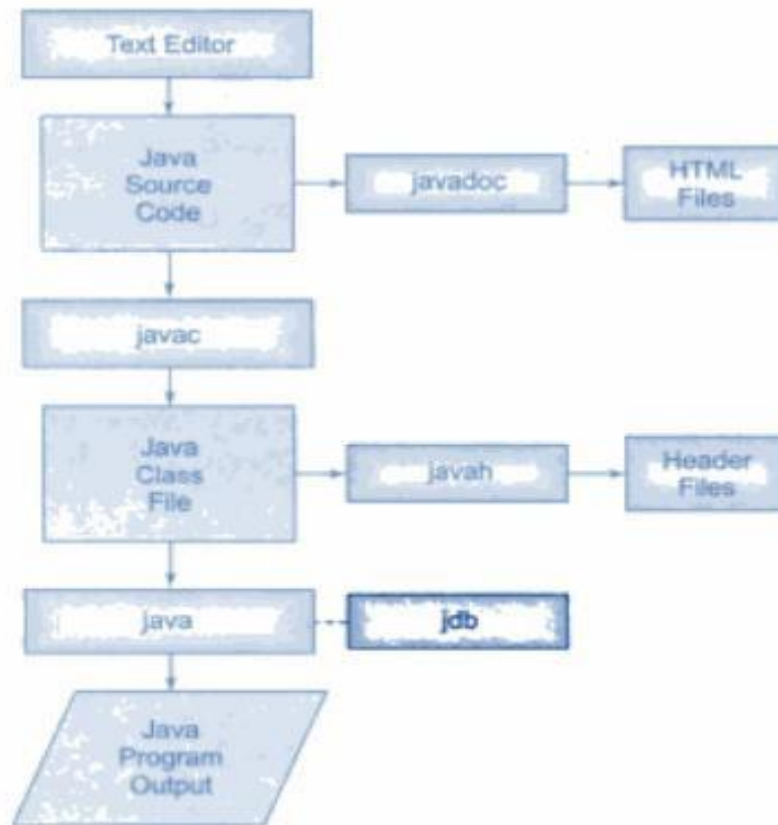


Fig. 2.5 *Process of building and running Java application programs*

Application Programming Interface

The Java Standard Library (or API) includes hundreds of classes and methods grouped into several functional packages (see Appendix G). Most commonly used packages are:

- **Language Support Package:** A collection of classes and methods required for implementing basic features of Java.
- **Utilities Package:** A collection of classes to provide utility functions such as date and time functions.
- **Input/Output Package:** A collection of classes required for input/output manipulation.
- **Networking Package:** A collection of classes for communicating with other computers via Internet.
- **AWT Package:** The Abstract Window Tool Kit package contains classes that implements platform-independent graphical user interface.
- **Applet Package:** This includes a set of classes that allows us to create Java applets.

The use of these library classes will become evident when we start developing Java programs.

Simple Java Program:

The best way to learn a new language is to write a few simple example programs and execute them. We begin with a very simple program that prints a line of text as output.

Program 3.1 A simple Java program

```
class SampleOne
{
    public static void main (String args[ ])
    {
        System.out.println("Java is better than C++.");
    }
}
```

Program 3.1 is perhaps the simplest of all Java programs. Nevertheless, it brings out some salient features of the language. Let us therefore discuss the program line by line and understand the unique features that constitute a Java program.

Class Declaration

The first line

class SampleOne

Declares a class, which is an object-oriented construct. As stated earlier, Java is a true object-oriented language and therefore, everything must be placed inside a class. `class` is a keyword and declares that a new class definition follows. `SampleOne` is a Java identifier that specifies the name of the class to be defined.

Opening Brace

Every class definition in Java begins with an opening brace "{" and ends with a matching closing brace "}", appearing in the last line in the example. This is similar to C++ class construct. (Note that a class definition in C++ ends with a semicolon.)

The Main Line

The third line.

public static void main(String args[])

defines a method named `main`. Conceptually, this is similar to the `main()` function in C/C++. Every Java application program must include the `main()` method. This is the starting point for the interpreter to begin the execution of the program. A Java application can have any number of classes but only one of them must include a `main` method to initiate the execution. (Note that Java applets will not use the `main` method at all.)

This line contains a number of keywords, `public`, `static` and `void`.

Public:	The keyword public is an access specifier that declares the main method as unprotected and therefore making it accessible to all other classes. This is similar to the C++ public modifier.
Static:	Next appears the keyword static , which declares this method as one that belongs to the entire class and not a part of any objects of the class. The main must always be declared as static since the interpreter uses this method before any objects are created. More about static methods and variables will be discussed later in Chapter 8.
Void:	The type modifier void states that the main method does not return any value (but simply prints some text to the screen.)

All parameters to a method are declared inside a pair of parentheses. Here, `String args[ ]` declares a parameter named `args`, which contains an array of objects of the class type `String`.

The Output Line

The only executable statement in the program is

```
System.out.println("Java is better than C++");
```

This is similar to the `printf()` statement of C or `cout << construct` of C++. Since Java is a true object oriented language, every method must be part of an object. The `println` method is a member of the `out` object, which is a static data member of `System` class. This line prints the string

```
Java is better than C++
```

to the screen. The method `println` always appends a newline character to the end of the string. This means that any subsequent output will start on a new line. Note the semicolon at the end of the statement. Every Java statement must end with a semicolon.

Java Program Structure

As we have seen in the previous examples, a Java program may contain many classes of which only one class defines a `main` method. Classes contain data members and methods that operate on the data members of the class. Methods may contain data type declarations and executable statements. To write a Java program, we first define classes and then put them together. A Java program may contain one or more sections as shown in Fig. 3.2.

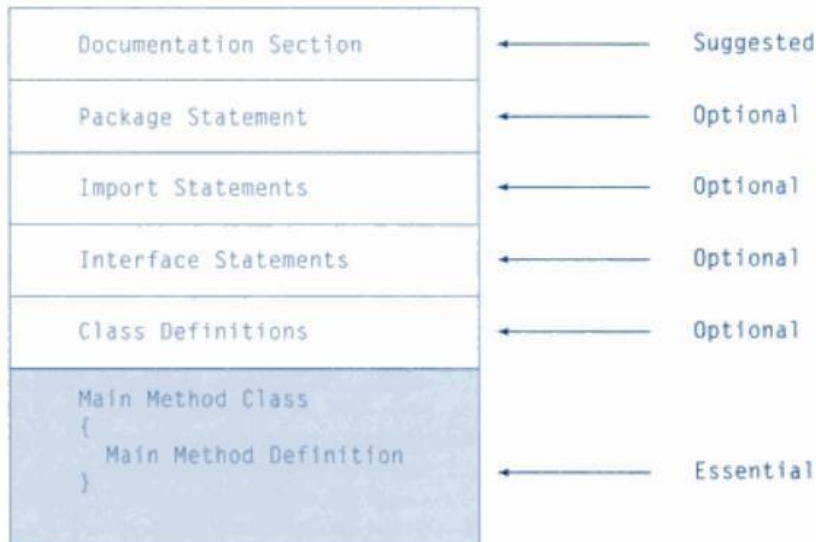


Fig. 3.2 General structure of a Java program

Documentation Section

The documentation section comprises a set of comment lines giving the name of the program, the author and other details, which the programmer would like to refer to at a later stage. Comments must explain why and what of classes and how of algorithms. This would greatly help in maintaining the program. In addition to the two styles of comments discussed earlier, Java also uses a third style of comment `//` known as documentation comment. This form of comment is used for generating documentation automatically.

Package Statement

The first statement allowed in a Java file is a package statement. This statement declares a package name and informs the compiler that the classes defined here belong to this package.

Example: `package student.`

The package statement is optional. That is, our classes do not have to be part of package.

Import Statements

The next thing after a package statement (but before any class definitions) may be a number of import statements. This is similar to the `#include` statement in C. Example:

`import student.test;`

This statement instructs the interpreter to load the test class contained in the package student. Using import statements, we can have access to classes that are part of other named packages.

Interface Statements

An interface is like a class but includes a group of method declarations. This is also an optional section and is used only when we wish to implement the multiple inheritance feature in the program.

Class Definitions

A Java program may contain multiple class definitions. Classes are the primary and essential elements of a Java program. These classes are used to map the objects of real-world problems. The number of classes used depends on the complexity of the problem.

Main Method Class

Since every Java stand-alone program requires a main method as its starting point, this class is the essential part of a Java program. A simple Java program may contain only this part. The main method creates objects of various classes and establishes communications between them. On reaching the end of main, the program terminates and the control passes back to the operating system.

Java Tokens:

A Java program is basically a collection of classes. A class is defined by a set of declaration statements and methods containing executable statements (see Fig. 3.3). Most statements contain expressions, which describe the actions carried out on data. Smallest individual units in a program are known as tokens. The compiler recognizes them for building up expressions and statements.

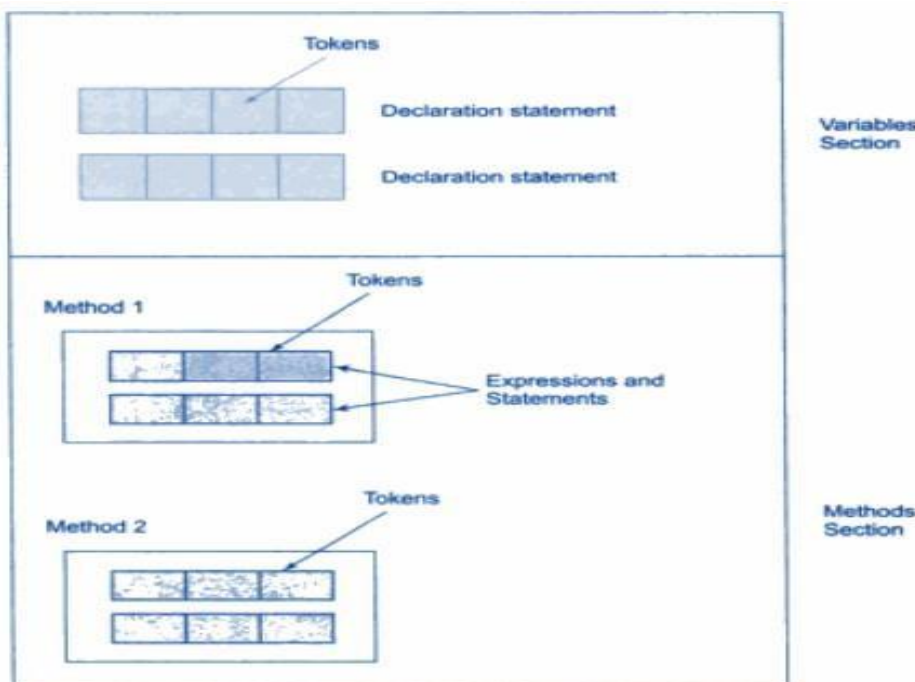


Fig. 3.3 Elements of Java class

In simple terms, a Java program is a collection of tokens, comments and white spaces. Java language includes five types of tokens. They are:

- Reserved Keywords
- Identifiers
- Literals
- Operators
- Separators

Java Character Set

The smallest units of Java language are the characters used to write Java tokens. These characters are defined by the Unicode character set, an emerging standard that tries to create characters for a large number of scripts worldwide.

The Unicode is a 16-bit character coding system and currently supports more than 34,000 defined characters derived from 24 languages from America, Europe, Middle East, Africa and Asia (including India). However, most of us use only the basic ASCII characters, which include letters, digits and punctuation marks, used in normal English. We, therefore, have used only ASCII character set (a subset of UNICODE character set) in developing the programs in this book.

Keywords

Keywords are an essential part of a language definition. They implement specific features of the language. Java language has reserved 50 words as keywords. Table 3.1 lists these keywords. These keywords, combined with operators and separators according to a syntax, form definition of the Java language. Understanding the meanings of all these words is important for Java programmers.

Since keywords have specific meaning in Java, we cannot use them as names for variables, classes, methods and so on. All keywords are to be written in lower-case letters. Since Java is case-sensitive, one can use these words as identifiers by changing one or more letters to upper case. However, it is a bad practice and should be avoided.

Table 3.1 Java Keywords			
abstract	assert	boolean	break
byte	case	catch	char
class	const	continue	default
do	double	else	enum
extends	final	finally	float
for	goto	if	implements
import	instanceof	int	interface
long	native	new	package
private	protected	public	return
short	static	strictfp	super
switch	synchronized	this	throw
throws	transient	try	void
volatile	while		

Note: We should also not attempt to use the boolean values true and false or null as names in our programs.

Identifiers

Identifiers are programmer-designed tokens. They are used for naming classes, methods, variables, objects, labels, packages and interfaces in a program. Java identifiers follow the following rules:

1. They can have alphabets, digits, and the underscore and dollar sign characters.
2. They must not begin with a digit.
3. Uppercase and lowercase letters are distinct.
4. They can be of any length.

Identifier must be meaningful, short enough to be quickly and easily typed and long enough to be descriptive and easily read. Java developers have followed some naming conventions.

- .Names of all public methods and instance variables start with a leading lowercase letter.

Examples:

average

sum

- When more than one word is used in a name, the second and subsequent words are marked with a leading uppercase letters. Examples:

dayTemperature

firstDayOfMonth

totalMarks

- All private and local variables use only lowercase letters combined with underscores. Examples:

length

batch_strength

- All classes and interfaces start with a leading uppercase letter (and each subsequent word with a leading uppercase letter). Examples:

Student

HelloJava

Vehicle

MotorCycle

- Variables that represent constant values use all uppercase letters and underscores between words. Examples:

TOTAL

F MAX

PRINCIPAL AMOUNT

They are like symbolic constants in C.

It should be remembered that all these are conventions and not rules. We may follow our own conventions as long as we do not break the basic rules of naming identifiers.

Literals

Literals in Java are a sequence of characters (digits, letters, and other characters) that represent constant values to be stored in variables. Java language specifies five major types of literals. They are:

- Integer literals
- Floating point literal
- Character literals
- String literals
- Boolean literals

Each of them has a type associated with it. The type describes how the values behave and how they are stored.

Operators

An operator is a symbol that takes one or more arguments and operates on them to produce a result.

Separators

Separators are symbols used to indicate where groups of code are divided and arranged. They basically define the shape and function of our code. Table 3.2 lists separators and their functions.

Table 3.2 Java Separators	
Name	What it is used for
parentheses ()	Used to enclose parameters in method definition and invocation, also used for defining precedence in expressions, containing expressions for flow control, and surrounding cast types.
braces { }	Used to contain the values of automatically initialized arrays and to define a block of code for classes, methods and local scopes
brackets []	Used to declare array types and for dereferencing array values
semicolon ;	Used to separate statements
comma ,	Used to separate consecutive identifiers in a variable declaration, also used to chain statements together inside a 'for' statement
period .	Used to separate package names from sub-packages and classes; also used to separate a variable or method from a reference variable.

Java Statements

The statements in Java are like sentences in natural languages. A statement is an executable combination of tokens ending with a semicolon (;) mark. Statements are usually executed in sequence in the order in which they appear. However, it is possible to control the flow of execution, if necessary, using special statements. Java implements several types of statements as illustrated in Fig. 3.4 and described in Table 3.3. They are considered in depth as and when they are encountered.

Table 3.3 Summary of Java Statements

Statement	Description	Remarks
Empty Statement	These do nothing and are used during program development as a place holder.	Same as C and C++
Labelled Statement	Any Statement may begin with a label. Such labels must not be keywords, already declared local variables or previously used labels in this module. Labels in Java are used as the arguments of Jump statements, which are described later in this list.	Identical to C and C++ except their use with jump statements
Expression Statement	Most statements are expression statements. Java has seven types of Expression statements: Assignment, Pre-Increment, Pre-Decrement, Post-Increment, Post-Decrement, Method Call and Allocation Expression.	Same as C++
Selection Statement	These select one of several control flows. There are Three types of selection statements in Java: if, if-else, and switch.	Same as C and C++

(Continued)

Table 3.3 (Continued)

Statement	Description	Remarks
Iteration Statement	These specify how and when looping will take place. There are three types of iteration statements; while, do and for.	Same as C and C++ except for jumps and labels
Jump Statement	Jump Statements pass control to the beginning or end of the current block, or to a labeled statement. Such labels must be in the same block, and continue labels must be on an iteration statement. The four types of Jump statement are break, continue, return and throw.	C and C++ do not use labels with jump statements
Synchronization Statement	These are used for handling issues with multithreading.	Now available in C and C++
Guarding Statement	Guarding statements are used for safe handling of code that may cause exceptions (such as division by zero). These statements use the keywords try, catch, and finally.	Same as in C++ except finally statement.

Java Virtual Machine

All language compilers translate source code into machine code for a specific computer. Java compiler also does the same thing. Then, how does Java achieve architecture neutrality? The answer is that the Java compiler produces an intermediate code known as bytecode for a machine that does not exist. This machine is called the Java Virtual Machine and it exists only inside the computer memory. It is a simulated computer within the computer and does all major functions of a real computer. Figure 3.6 illustrates the process of compiling a Java program into bytecode which is also referred to as virtual machine code.

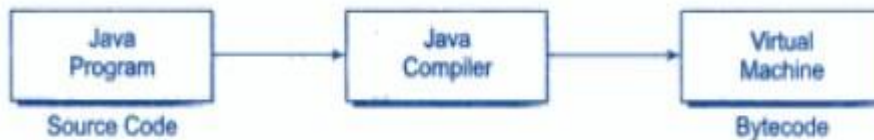


Fig. 3.6 *Process of compilation*

The virtual machine code is not machine specific. The machine specific code (known as machine code) is generated by the Java interpreter by acting as an intermediary between the virtual machine and the real machine as shown in Fig. 3. Remember that the interpreter is different for different machines.



Fig. 3.7 *Process of converting bytecode into machine code*

Figure 3.8 illustrates how Java works on a typical computer. The Java object framework (Java API) acts as the intermediary between the user programs and the virtual machine which in turn acts as the intermediary between the operating system and the Java object framework.

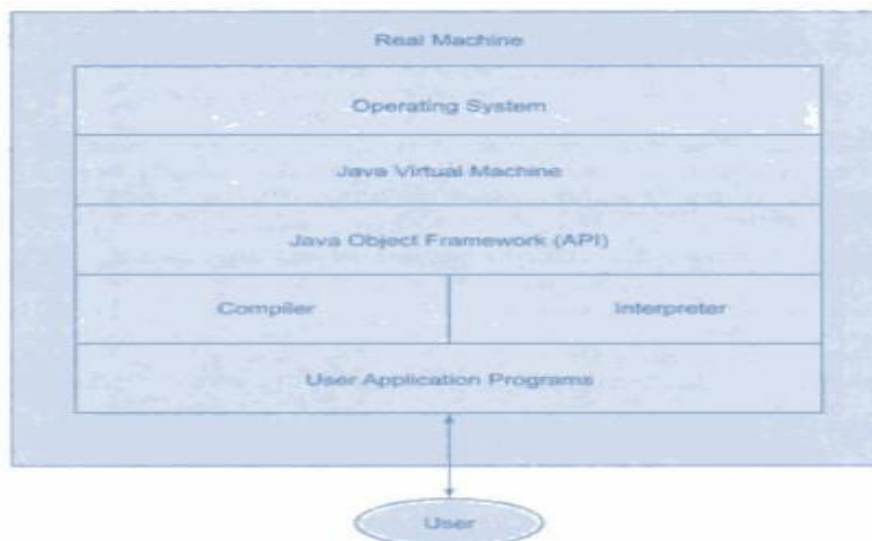


Fig. 3.8 *Layers of interactions for Java programs*

Chapter 3 Java Programming Basics

Constants:

Constants in Java refer to fixed values that do not change during the execution of a program. Java supports several types of constants as illustrated in Fig. 4.1.

Integer Constants

An integer constant refers to a sequence of digits. There are three types of integers, namely, decimal integer, octal integer and hexadecimal integer.

Decimal integers consist of a set of digits, 0 through 9, preceded by an optional minus sign. Valid examples of decimal integer constants are:

123 -321 0 654321

Embedded spaces, commas, and non-digit characters are not permitted between digits. For example,

1 5 7 50 20.000 \$1000

are illegal numbers.

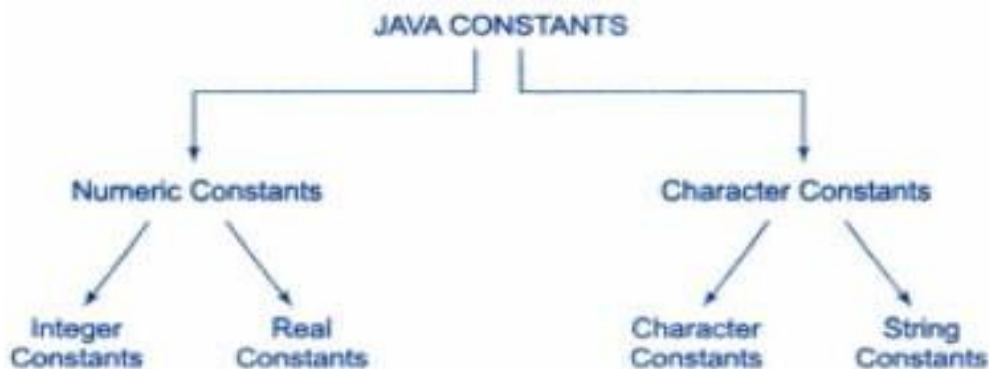


Fig. 4.1 Java constants

An octal integer constant consists of any combination of digits from the set 0 through 7, with a leading 0. Some examples of octal integer are:

037 0 0435 0551

A sequence of digits preceded by 0x or 0X is considered as hexadecimal integer (hex integer). They may also include alphabets A through F or a through f. A letter A through F represents the numbers 10 through 15. Following are the examples of valid hex integers.

0X2 0X9F 0cd 0x

We rarely use octal and hexadecimal numbers in programming.

Real Constants

Integer numbers are inadequate to represent quantities that vary continuously, such as distances, heights, temperatures, prices, and so on. These quantities are represented by numbers containing fractional parts like 17.548. Such numbers are called real (or floating point) constants. Further examples of real constants are:

0.0083 -0.75 435.36

These numbers are shown in decimal notation, having a whole number followed by a decimal point and the fractional part, which is an integer. It is possible that the number may not have digits before the decimal point or digits after the decimal point. That is,

215. 95 -.71

are all valid real numbers.

A real number may also be expressed in exponential (or scientific) notation. For example, the value 215.65 may be written as 2.1565e2 in exponential notation. e2 means multiply by 10. The general form is:

mantissa e exponent

The mantissa is either a real number expressed in decimal notation or an integer. The exponent is an integer with an optional plus or minus sign. The letter e separating the mantissa and the exponent can be written in either lowercase or uppercase. Since the exponent causes the decimal point to "float", this notation is said to represent a real number in floating point form. Examples of legal floating point constants are:

0.65e4 12e-2 1.5e +5 3.18E3 -1.2E-1

Embedded white (blank) space is not allowed, in any numeric constant.

Exponential notation is useful for representing numbers that are either very large or very small in magnitude. For example, 7500000000 may be written as 7.5E9 or 75E8. Similarly, -0.000000368 is equivalent to -3.68E-7.

A floating point constant may thus comprise four parts:

- a whole number
- a decimal point
- a fractional part

- an exponent

Single Character Constants

A single character constant (or simply character constant) contains a single character enclosed within a pair of single quote marks. Examples of character constants are:

'5' 'X' ';' ' '

Note that the character constant '5' is not the same as the number 5. The last constant is a blank space.

String Constants

A string constant is a sequence of characters enclosed between double quotes. The characters may be alphabets, digits, special characters and blank spaces. Examples are:

"Hello Java" "1997" "WELL DONE" "2...!" "5+3" "X"

Backslash Character Constants

Java supports some special backslash character constants that are used in output methods. For example, the symbol "\n" stands for newline character. A list of such backslash character constants is given in Table 4.1. Note that each one of them represents one character, although they consist of two characters. These characters combinations are known as escape sequences.

Table 4.1 Backslash Character Constants	
Constant	Meaning
'\b'	back space
'\f'	form feed
'\n'	new line
'\r'	carriage return
'\t'	horizontal tab
'\''	single quote
'\"'	double quote
'\\'	backslash

Variables

A variable is an identifier that denotes a storage location used to store a data value. Unlike constants that remain unchanged during the execution of a program, a variable may take different values at different times during the execution of the program. In Chapter 3, we had used several variables. For instance, we used variables length and breadth to store the values of length and breadth of a room.

A variable name can be chosen by the programmer in a meaningful way so as to reflect what it represents in the program, Some examples of variable names are:

- Average
- Height
- total_height
- classStrength

As mentioned earlier, variable names may consist of alphabets, digits, the underscore(_) and dollar characters, subject to the following conditions:

1. They must not begin with a digit.
2. Uppercase and lowercase are distinct. This means that the variable Total is not the same as total or TOTAL.
3. It should not be a keyword.
4. White space is not allowed.
5. Variable names can be of any length.

Data Types

Every variable in Java has a data type. Data types specify the size and type of values that can be stored. Java language is rich in its data types. The variety of data types available allow the programmer to select the type appropriate to the needs of the application. Data types in Java under various categories are shown in Fig. 4.2. Primitive types (also called intrinsic or built-in types) are discussed in detail in

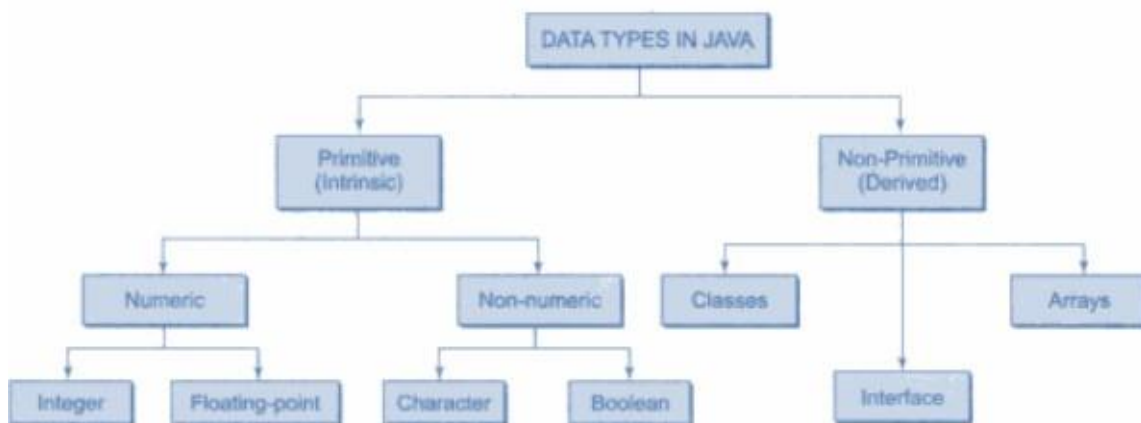


Fig. 4.2 Data types in Java

this chapter. Derived types (also known as reference types) are discussed later as and when they are encountered.

Integer Types

Integer types can hold whole numbers such as 123, -96, and 5639. The size of the values that can be stored depends on the integer data type we choose. Java supports four types of integers as shown in Fig. 4.3. They are byte, short, int, and long. Java does not support the concept of unsigned types and therefore all Java values are signed meaning they can be positive or negative. Table 4.3 shows the memory size and range of all the four integer data types.

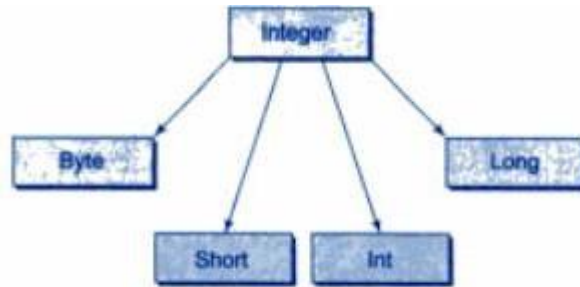


Fig. 4.3 Integer data types

Table 4.3 Size and Range of Integer Types			
Type	Size	Minimum value	Maximum value
byte	One byte	-128	127
short	Two bytes	-32, 768	32, 767
int	Four bytes	-2, 147, 483, 648	2, 147, 483, 647
long	Eight bytes	-9, 223, 372, 036, 854, 775, 808	9, 223, 372, 036, 854, 775, 807

It should be remembered that wider data types require more time for manipulation and therefore it is advisable to use smaller data types, wherever possible. For example, instead of storing a number like 50 in an int type variable, we must use a byte variable to handle this number. This will improve the speed of execution of the program.

We can make integers long by appending the letter L or l at the end of the number. Example:

123L or 123l

Floating Point Types

Integer types can hold only whole numbers and therefore we use another type known as floating point type to hold numbers containing fractional parts such as 27.59 and -1.375 (known as floating point constants). There are two kinds of floating point storage in Java as shown in Fig. 4.4.

The float type values are single-precision numbers while the double types represent double precision numbers. Table 4.4 gives the size and range of these two types.

Floating point numbers are treated as double-precision quantities. To force them to be in single precision mode, we must append f or F to the numbers. Example:

```
1.23f  
7.56923e5F
```

Table 4.4 Size and Range of Floating Point Types

Type	Size	Minimum value	Maximum value
float	4 bytes	3.4e-038	3.4e+038
double	8 bytes	1.7e-308	1.7e+308

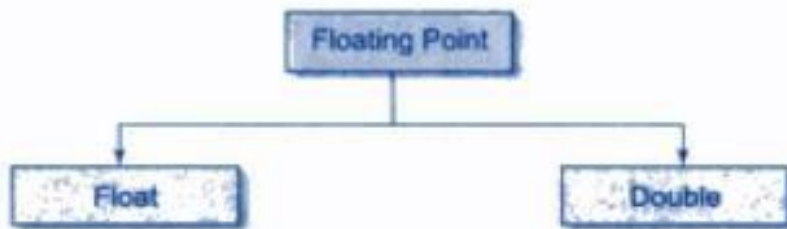


Fig. 4.4 Floating point data types

Double-precision types are used when we need greater precision in storage of floating point numbers. All mathematical functions, such as sin, cos and sqrt return double type values.

Floating point data types support a special value known as Not-a-Number (NaN). NaN is used to represent the result of operations such as dividing zero by zero, where an actual number is not produced. Most operations that have NaN as an operand will produce NaN as a result.

Character Type

In order to store character constants in memory, Java provides a character data type called char. The char type assumes a size of 2 bytes but, basically, it can hold only a single character.

Boolean Type

Boolean type is used when we want to test a particular condition during the execution of the program. There are only two values that a boolean type can take: true or false. Boolean type is denoted by the keyword boolean and uses only one bit of storage.

All comparison operators return boolean type values. Boolean values are often used in selection and iteration statements. The words true and false cannot be used as identifiers.

Declaration of variables

In java, variables are the names of storage locations. After designing suitable variable names , we must declare them to the compiler. Declaration does three things:

1. It tells the compiler what the variable name is.
2. It specifies what type of the data variable will hold.
3. The place of declaration decides the scope of variable.

A variable must be declared before it is used in the program.

A variable can be used to store a value of any data type. That is, the name has nothing to do with the type. Java allows any properly formed variable to have any declared data type. The declaration statements defines the type of variable. The general form of declaration of a variable is:

```
type variable1, variable2, ..... variableN;
```

Variable are separated by commas. A declaration statement must end with a semicolon. Some valid declarations are:

```
int      count:
float    x, y:
double   pi:
byte     b:
char     c1, c2, c3:
```

Giving values to the variable

A variable must be given a value after it has been declared but before it is used in an expression. This can be achieved in two ways:

1. By using an assignment statement.
2. By using a read statement.

Assignment Statement

A statement method of giving value to a variable is through the assignment statement as follows:

```
variableName = value;
```

For example

```
initialValue = 0;
finalValue   = 100;
yes          = 'x';
```

We can also string assignment expression as shown below:

```
X=Y=Z=0;
```

It is also possible to assign a value to a variable at the time of its declaration. This takes the form:

```
type variableName = value;
```

Example:

```
int    finalValue = 100;
char   yes        = 'x';
double total      = 75.36;
```

The process of giving initial values to variables is known as the initialization. The one that are not initialized are automatically set to zero.

The following are valid java statements:

```
float x, y, z;           // declares three float variables
int   m = 5, n = 10;     // declares and initialises two int variables
int   m, n = 10;         // declares m and n and initialises n
```

Read Statement

We may also give values to variable interactively through the keyboard using the `readLine()` method illustrated in program 4.1

The `readLine()` method (which is invoked using an object of the class `DataInputStream`) reads the input from the keyboard as a string which is then converted to the corresponding data type using the data type wrapper classes.

Note that we have used the keywords `try` and `catch` to handle any errors that might occur during the reading process. Java requires this.

Program 4.1 Reading data from keyboard

```
import java.io.DataInputStream;
class Reading
{
    public static void main(String args[])
    {
        DataInputStream in = new DataInputStream(System.in);
        int intNumber = 0;
        float floatNumber = 0.0f;

        try
        {
            System.out.println("Enter an Integer: ");
            intNumber = Integer.parseInt(in.readLine());
            System.out.println("Enter a float number: ");
            floatNumber =
                Float.valueOf(in.readLine()).floatValue();
        }
        catch (Exception e) { }

        System.out.println("intNumber = " + intNumber);
        System.out.println("floatNumber = " + floatNumber);
    }
}
```

The interactive input and output of Program 4.1 are shown below:

```
Enter an integer:
123
Enter a float number:
123.45
intNumber = 123
floatNumber = 123.45
```

Scope of variables

Java variables are actually classified into three kinds:

- instance variables,
- class variables, and
- local variables.

Instance and class variables are declared inside a class. Instance variables are created when the objects are instantiated and therefore they are associated with the objects. They take different values for each object. On the other hand, class variables are global to a class and belong to the entire set of objects that class creates. Only one memory location is created for each class variable..

Variables declared and used inside methods are called local variables. They are called so because they are not available for use outside the method definition. Local variables can also be declared

inside program blocks that are defined between an opening brace { and a closing brace). These variables are visible to the program only from the beginning of its program block to the end of the program block. When the program control leaves a block, all the variables in the block will cease to exist. The area of the program where the variable is accessible (i.e., usable) is called its scope.

We can have program blocks within other program blocks (called nesting) as shown in Fig. 4.5.

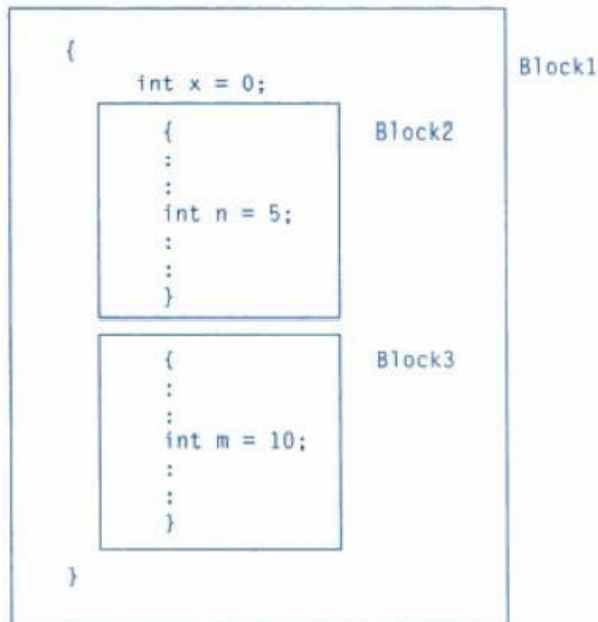


Fig. 4.5 Nested program blocks

Each block can contain its own set of local variable declarations. We cannot, however, declare a variable to have the same name as one in an outer block. In Fig. 4.5, the variable *x* declared in Block1 is available in all the three blocks. However, the variable *n* declared in Block2 is available only in Block2, because it goes out of the scope at the end of Block2. Similarly, *m* is accessible only in Block3. Note that we cannot declare the variable *x* again in Block2 or Block3 (This is perfectly legal in C and C++).

Symbolic Constants

We often use certain unique constants in a program. These constants may appear repeatedly in a number of places in the program. One example of such a constants is 3.142, representing the value of the mathematical constant "pi". Another example is the total number of students whose mark-sheets are analysed by a 'test analysis program'. The number of students, say 50, may be used for calculating the class total, class average, standard deviation, etc. We face two problems in the subsequent use of such programs. They are:

1. Problem in modification of the program.
2. Problem in understanding the program.

Modifiability

We may like to change the value of "pi" from 3.142 to 3.14159 to improve the accuracy of calculations or the number 50 to 100 to process the test results of another class. In both the cases, we will have to search throughout the program and explicitly change the value of the constant wherever it has been used. If any value is left unchanged, the program may produce disastrous outputs.

Understandability

When a numeric value appears in a program, its use is not always clear, especially when the same value means different things in different places. For example, the number 50 may mean the number of students at one place and the 'pass marks' at another place of the same program. We may forget what a certain number meant, when we read the program some days later.

Assignment of a symbolic name to such constants frees us from these problems. For example, we may use the name **STRENGTH** to denote the number of students and **PASS_MARK** to denote the pass marks required in a subject. Constant values are assigned to these names at the beginning of the program. Subsequent use of the names **STRENGTH** and **PASS_MARK** in the program has the effect of causing their defined values to be automatically substituted at the appropriate points. A constant is declared as follows:

```
final type symbolic-name = value;
```

Valid example for constant declarations are:

```
final int    STRENGTH = 100;
final int    PASS_MARK = 50;
final float  PI = 3.14159;
```

Note that:

1. Symbolic names take the same form as variable names. But, they are written in CAPITALS to visually distinguish them from normal variable names. This is only a convention, not a rule.
2. After declaration of symbolic constants, they should not be assigned any other value within the program by using an assignment statement. For example, STRENGTH=200, is illegal.
3. Symbolic constants are declared for types. This is not done in C and C++ where symbolic constants are defined using the # define statement.
4. They can NOT be declared inside a method. They should be used only as class data members in the beginning of the class.

Type Casting

We often encounter situations where there is a need to store a value of one type into a variable of another type. In such situations, we must cast the value to be stored by preceding it with the type name in parentheses. The syntax is:

```
type variable1 = (type) variable2;
```

The process of converting one data type to another is called casting.

Examples:

```
int    m    = 50;
byte   n    = (byte)m;
long   count = (long)m;
```

Casting is often necessary when a method returns a type different than the one we require.

Four integer types can be cast to any other type except boolean. Casting into a smaller type may result in a loss of data. Similarly, the float and double can be cast to any other type except boolean. Again, casting to smaller type can result in a loss of data. Casting a floating point value to an integer will result in a loss of the fractional part. Table 4.5 lists those casts, which are guaranteed to result in no loss of information.

Table 4.5 Casts that Results in No Loss of Information

From	To
byte	short, char, int, long, float, double
short	int, long, float, double
char	int, long, float, double
int	long, float, double
long	float, double
float	double

Automatic Conversion

For some types, it is possible to assign a value of one type to a variable of a different type without a cast. Java does the conversion of the assigned value automatically. This is known as automatic type conversion. Automatic type conversion is possible only if the destination type has enough precession to store the source valls. For example, int is large enough to hold a byte value. Therefore,

```
byte b= 75;
```

```
int  a= b;
```

are valid statements.

The process of assigning a smaller type to a larger one is known as widening or promotion and that of assigning a larger type to a smaller one is known as narrowing. Note that narrowing may result in

loss of information. Program 4.2 illustrates the creation of variables of basic types and also shows the effect of type conversions.

Program 4.2 Creation and casting of variables

```
class TypeWrap
{
    public static void main(String args[])
    {
        System.out.println("Variables created");
        char c = 'x';
        byte b = 50;
        short s = 1996;
        int i = 123456789;
        long l = 1234567654321L;
        float f1 = 3.142F;
        float f2 = 1.2e-5F;
        double d2 = 0.000000987;

        System.out.println(" c = " + c);
        System.out.println(" b = " + b);
        System.out.println(" s = " + s);
        System.out.println(" i = " + i);
        System.out.println(" l = " + l);
        System.out.println(" f1 = " + f1);
        System.out.println(" f2 = " + f2);
        System.out.println(" d2 = " + d2);

        System.out.println(" ");
        System.out.println("Types converted");
        short s1 = (short)b;
        short s2 = (short)i;    // Produces incorrect result
        float n1 = (float)l;
        int m1 = (int)f1;       // Fractional part is lost

        System.out.println(" (short)b = " + s1);
        System.out.println(" (short)i = " + s2);
        System.out.println(" (float)l = " + n1);
        System.out.println(" (int) f1 = " + m1);
    }
}
```

Output of Program 4.2 is as follows:

```
Variables created
c = x
b = 50
s = 1996
i = 123456789
l = 1234567654321
f1 = 3.142
f2 = 1.2e-005
d2 = 9.87e-007

Types converted
(short)b = 50
(short)i = -13035
(float)l = 1.23457e+012
(int)f1 = 3
```

Note that floating point constants have a default type of double. What happens when we want to declare a float variable and initialize it using a constant? Example:

```
float x= 7.56;
```

This will cause the following compiler error;

Incompatible type for declaration. Explicit cast needed to convert double to float.

This should be written as:

```
float x 7.56F;
```

Chapter 4 : Operators and Expressions

Introduction

Java supports a rich set of operators. We have already used several of them, such as =, +, -, and *. An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations. Operators are used in programs to manipulate data and variables. They usually form a part of mathematical or logical expressions.

Java operators can be classified into a number of related categories as below:

1. Arithmetic operators
2. Relational operators
3. Logical operators
4. Assignment operators
5. Increment and decrement operators
6. Conditional operators
7. Bitwise operators
8. Special operators

In this chapter, we discuss each one of these categories with illustrations.

Arithmetic Operators

Arithmetic operators are used to construct mathematical expressions as in algebra. Java provides all the basic arithmetic operators. They are listed in Table 5.1. The operators +, -, and / all work the same way as they do in other languages. These can operate on any built-in numeric data type of Java. We cannot use these operators on boolean type. The unary minus operator, in effect, multiplies its single operand by - 1. Therefore, a number preceded by a minus sign changes its sign.

Table 5.1 Arithmetic Operators	
Operator	Meaning
+	Addition or unary plus
-	Subtraction or unary minus
*	Multiplication
/	Division
%	Modulo division (Remainder)

Arithmetic operators are used as shown below:

a-b a+b

a* b a/b

$a \% b$ $-a * b$

Here a and b may be variables or constants and are known as operands.

Integer Arithmetic

When both the operands in a single arithmetic expression such as $a + b$ are integers, the expression is called an integer expression, and the operation is called integer arithmetic. Integer arithmetic always yields an integer value. In the above examples, if a and b are integers, then for $a = 14$ and $b = 4$ we have the following results:

$a - b = 10$

$a + b = 18$

$a * b = 56$

$a / b = 3$ (decimal part truncated)

$a \% b = 2$ (remainder of integer division)

a / b , when a and b are integer types, gives the result of division of a by b after truncating the divisor.

This operation is called the integer division.

For modulo division, the sign of the result is always the sign of the first operand (the dividend). That is

$-14 \% 3 = -2$

$-14 \% -3 = -2$

$14 \% 3 = 2$

(Note that modulo division is defined as: $a \% b = a - (a / b) * b$, where a / b is the integer division.)

Real Arithmetic

An arithmetic operation involving only real operands is called real arithmetic. A real operand may assume values either in decimal or exponential notation. Since floating point values are rounded to the number of significant digits permissible, the final value is an approximation of the correct result.

Unlike C and C++, modulus operator $\%$ can be applied to the floating point data as well. The floating point modulus operator returns the floating point equivalent of an integer division. What this means is that the division is carried out with both floating point operands, but the resulting divisor is treated as an integer, resulting in a floating point remainder. Program 5.1 shows how arithmetic operators work on floating point values.

Program 5.1 Floating point arithmetic

```
class FloatPoint
{
    public static void main(String args[])
    {
        float a = 20.5F, b = 6.4F;
        System.out.println(" a = " + a);
        System.out.println(" b = " + b);
        System.out.println(" a+b = " + (a+b));
        System.out.println(" a-b = " + (a-b));
        System.out.println(" a*b = " + (a*b));
        System.out.println(" a/b = " + (a/b));
        System.out.println(" a%b = " + (a%b));
    }
}
```

The output of Program 5.1 is as follows:

```
a = 20.5
b = 6.4
a+b = 26.9
a-b = 14.1
a*b = 131.2
a/b = 3.20313
a%b = 1.3
```

Mixed-mode Arithmetic

When one of the operands is real and the other is integer, the expression is called a mixed-mode arithmetic expression. If either operand is of the real type, then the other operand is converted to real and the real arithmetic is performed. The result will be a real. Thus

15/10.0 produces the result 1.5

whereas

15/10 produces the result

More about mixed operations will be discussed later when we deal with the evaluation of expressions.

Relational Operators

We often compare two quantities, and depending on their relation, take certain decisions. For example, we may compare the age of two persons, or the price of two items, and so on. These comparisons can be done with the help of relational operators. We have already used the symbol meaning 'less than'. An expression such as

$a < b$ or $x < 20$

containing a relational operator is termed as a relational expression. The value of relational expression is either true or false. For example, if $x=10$, then

$x < 20$ is true

while

$20 < x$ is false.

Java supports six relational operators in all. These operators and their meanings are shown in Table 5.2.

Table 5.2 Relational Operators	
Operator	Meaning
<	is less than
<=	is less than or equal to
>	is greater than
>=	is greater than or equal to
==	is equal to
!=	is not equal to

A simple relational expression contains only one relational operator and is of the following form:

ae-1 relational operator ae-2

ae-1 and ae-2 are arithmetic expressions, which may be simple constants, variables or combination of them. Table 5.3 shows some examples of simple relational expressions and their values.

Table 5.3 Relational Expressions	
Expression	Value
$4.5 \leq 10$	TRUE
$4.5 < -10$	FALSE
$-35 \geq 0$	FALSE
$10 < 7 + 5$	TRUE
$a + b == c + d$	TRUE*

When arithmetic expressions are used on either side of a relational operator, the arithmetic expressions will be evaluated first and then the results compared. That is, arithmetic operators have a higher priority over relational operators. Program 5.2 shows the implementation of relational operators.

Program 5.2 Implementation of relational operators

```
class RelationalOperators
{
    public static void main(String args[])
    {
        float a = 15.0F, b = 20.75F, c = 15.0F;
        System.out.println(" a = " + a);
        System.out.println(" b = " + b);
        System.out.println(" c = " + c);
        System.out.println(" a < b is " + (a<b));
        System.out.println(" a > b is " + (a>b));
        System.out.println(" a == c is " + (a==c));
        System.out.println(" a <= c is " + (a<=c));
        System.out.println(" a >= b is " + (a>=b));
        System.out.println(" b != c is " + (b!=c));
        System.out.println(" b == a+c is " + (b==a+c));
    }
}
```

The output of Program 5.2 would be:

```
a = 15
b = 20.75
c = 15
a < b is true
a > b is false
a == c is true
a <= c is true
a >= b is false
a != c is true
b == a+c is false
```

Relational expressions are used in decision statements such as, if and while to decide the course of action of a running program.

Logical Operators

In addition to the relational operators, Java has three logical operators, which are given in Table 5.4.

Table 5.4 Logical Operators	
Operator	Meaning
&&	logical AND
	logical OR
!	logical NOT

The logical operators && and || are used when we want to form compound conditions by combining two or more relations. An example is:

a > b && x==10

An expression of this kind which combines two or more relational expressions is termed as a logical expression or a compound relational expression. Like the simple relational expressions, a logical expression also yields a value of true or false, according to the truth table shown in Table 5.5. The logical expression given above is true only if both $a > b$ and $x=10$ are true. If either (or both) of them are false the expression is false.

Table 5.5 Truth Table				
<i>op-1</i>	<i>op-2</i>	<i>Value of the expression</i>		
		<i>op-1 && op-2</i>	<i>op-1 op-2</i>	
true	true	true	true	
true	false	false	true	
false	true	false	true	
false	false	false	false	

Note:

- $op-1 \ \&\& \ op-2$ is true if both $op-1$ and $op-2$ are true and false otherwise.
- $op-1 \ || \ op-2$ is false if both $op-1$ and $op-2$ are false and true otherwise.

Some examples of the usage of logical expressions are:

1. `if (age 55 && salary<1000)`
2. `if (number<0 || number>100)`

Assignment Operator

Assignment operators are used to assign the value of an expression to a variable. We have seen the usual assignment operator, `*=`. In addition, Java has a set of 'shorthand' assignment operators which are used in the form

$$v \ op = \ exp;$$

where v is a variable, exp is an expression and op is a Java binary operator. The operator $op =$ is known as the shorthand assignment operator. The assignment statement

$v \ op \ exp;$

is equivalent to

$v = v \ op(exp);$

with v accessed only once. Consider an example

$x += y+1;$

This is same as the statement

$X = x+(y+1);$

The shorthand operator `+=` means 'add $y+1$ to x ' or 'increment x by $y+1$ '. For $y = 2$, the above statement becomes

`x += 3;`

and when this statement is executed, 3 is added to x. If the old value of x is, say 5, then the new value of x is 8. Some of the commonly used shorthand assignment operators are illustrated in Table 5.6.

Table 5.6 Shorthand Assignment Operators	
<i>Statement with simple assignment operator</i>	<i>Statement with shorthand operator</i>
<code>a = a+1</code>	<code>a += 1</code>
<code>a = a-1</code>	<code>a -= 1</code>
<code>a = a*(n+1)</code>	<code>a *= n+1</code>
<code>a = a/(n+1)</code>	<code>a /= n+1</code>
<code>a = a%b</code>	<code>a %= b</code>

The use of shorthand assignment operators has three advantages:

1. What appears on the left-hand side need not be repeated and therefore it becomes easier to write.
2. The statement is more concise and easier to read.
3. Use of shorthand operator results in a more efficient code.

Increment and Decrement Operators

Java has two very useful operators not generally found in many other languages. These are the increment and decrement operators:

`++` and `--`

The operator `++` adds 1 to the operand while `--` subtracts 1. Both are unary operators and are used in the following form:

```
++m; or m++;  
--m; or m--;  
++m; is equivalent to m = m + 1; (or m += 1;)  
--m; is equivalent to m = m - 1; (or m -= 1;)
```

We use the increment and decrement operators extensively in for and while loops. (See Chapter 7.) While `++m` and `m++` mean the same thing when they form statements independently, they behave differently when they are used in expressions on the right-hand side of an assignment statement. Consider the following:

`m = 5;`

`y = ++ m`

In this case, the value of y and m would be 6. Suppose, if we rewrite the above statement as

`m = 5;`

`y = m+ +`

then, the value of y would be 5 and m would be 6. A prefix operator first adds 1 to the operand and then the result is assigned to the variable on left. On the other hand, a postfix operator first assigns the value to the variable on left and then increments the operand. Program 5.3 illustrates this.

Program 5.3 Increment Operator Illustrated

```
class IncrementOperator
{
    public static void main(String args[])
    {
        int m = 10, n = 20;
        System.out.println(" m = " + m);
        System.out.println(" n = " + n);
        System.out.println(" ++m = " + ++m);
        System.out.println(" n++ = " + n++);
        System.out.println(" m = " + m);
        System.out.println(" n = " + n);
    }
}
```

Output of Program 5.3 is as follows:

```
m = 10
n = 20
++m = 11
n++ = 20
m = 11
n = 21
```

Similar is the case, when we use ++ (or --) in subscripted variables. That is, the statement

```
a[i++] = 10
```

is equivalent to

```
a[i] = 10
i = i+1
```

Conditional Operator

The character pair ?: is a ternary operator available in Java. This operator is used to construct conditional expressions of the form

$$exp1 \ ? \ exp2 \ : \ exp3$$

where exp1, exp2, and exp3 are expressions.

The operator?: works as follows: exp1 is evaluated first. If it is nonzero (true), then the expression exp2 is evaluated and becomes the value of the conditional expression. If exp1 is false, exp3 is evaluated and its Value becomes the value of the conditional expression. Note that only one of the expressions (either exp2 or exp3) is evaluated. For example, consider the following statements:

```
a = 10;
```

```
b = 15;
```

```
x-(a > b) ? a: b;
```

In this example, x will be assigned the value of b. This can be achieved using the if...else statement

as follows:

```
if(a > b)
```

```
    x = a;
```

```
else
```

```
    x = b;
```

Special Operators

Java supports some special operators of interest such as instanceof operator and member selection operator (.).

Instanceof Operator

The instanceof is an object reference operator and returns true if the object on the left-hand side is an instance of the class given on the right-hand side. This operator allows us to determine whether the object belongs to a particular class or not.

Example:

```
Person      instanceof      student
```

is true if the object, person belongs to the class student; otherwise it is false.

Dot Operator

The dot operator (.) is used to access the instance variables and methods of class objects.

Examples:

```
Person1. Age // Reference to the variable age
```

```
person1.salary() // Reference to the method salary()
```

It is also used to access classes and sub-packages from a package.

Mathematical functions

Mathematical functions such as cos, sqrt, log, etc. are frequently used in analysis of real-life problems. Java supports these basic math functions through Math class defined in the java.lang package. Table 5.12 lists the math functions defined in the Math class. These functions should be used as follows:

Math. function_name()

Example:

```
double y = Math.sqrt(x);
```

Table 5.12 Math Functions	
Functions	Action
sin(x)	Returns the sine of the angle x in radians
cos(x)	Returns the cosine of the angle x in radians
tan(x)	Returns the tangent of the angle x in radians
asin(y)	Returns the angle whose sine is y
acos(y)	Returns the angle whose cosine is y
atan(y)	Returns the angle whose tangent is y
atan2(x,y)	Returns the angle whose tangent is x/y
pow(x,y)	Returns x raised to y (x^y)
exp(x)	Returns e raised to x (e^x)
log(x)	Returns the natural logarithm of x
sqrt(x)	Returns the square root of x
ceil(x)	Returns the smallest whole number greater than or equal to x. (Rounding up)
floor(x)	Returns the largest whole number less than or equal to x (Rounded down)
rint(x)	Returns the truncated value of x.
round(x)	Returns the integer closest to the argument
abs(a)	Returns the absolute value of a
max(a,b)	Returns the maximum of a and b
min(a,b)	Returns the minimum of a and b

Chapter 5 : Using I/O

Byte streams and character streams

Modern versions of Java define two types of streams: byte and character. (The original version of Java defined only the byte stream, but character streams were quickly added.) Byte streams provide a convenient means for handling input and output of bytes. They are used, for example, when reading or writing binary data. They are especially helpful when working with files. Character streams are designed for handling the input and output of characters. They use Unicode and, therefore, can be internationalized. Also, in some cases, character streams are more efficient than byte streams.

The fact that Java defines two different types of streams makes the I/O system quite large because two separate sets of class hierarchies (one for bytes, one for characters) are needed. The sheer number of I/O classes can make the I/O system. For the most part, the functionality of byte streams is paralleled by that of the character streams. At the lowest level, all I/O is still byte-oriented. The character-based streams simply provide a convenient and efficient means for handling characters.

The Byte Stream Classes:

Byte streams are defined by using two class hierarchies. At the top of these are two abstract classes: `InputStream` and `OutputStream`. `InputStream` defines the characteristics common to byte input streams and `OutputStream` describes the behavior of byte output streams. From `InputStream` and `OutputStream` are created several concrete subclasses that offer varying functionality and handle the details of reading and writing to various devices, such as disk files. The byte stream classes are shown in Table 3.1

Byte Stream Class	Meaning
<code>BufferedInputStream</code>	Buffered input stream
<code>BufferedOutputStream</code>	Buffered output stream
<code>ByteArrayInputStream</code>	Input stream that reads from a byte array
<code>ByteArrayOutputStream</code>	Output stream that writes to a byte array
<code>DataInputStream</code>	An input stream that contains methods for reading the java standard data types
<code>DataOutputStream</code>	An output stream that contains methods for writing the java standard data types
<code>FileInputStream</code>	Input stream that reads from a file
<code>FileOutputStream</code>	Output stream that writes to a file
<code>FilterInputStream</code>	Implements <code>InputStream</code>
<code>FilterOutputStream</code>	Implements <code>OutputStream</code>
<code>InputStream</code>	Abstract class that describes stream input
<code>ObjectInputStream</code>	Input Stream for objects
<code>ObjectOutputStream</code>	Output Stream for objects
<code>OutputStream</code>	Abstract class that describes stream output
<code>PipedInputStream</code>	Input pipe
<code>PipedOutputStream</code>	Output pipe
<code>PrintStream</code>	Output Stream that contains <code>print()</code> and <code>println()</code>
<code>PushbackInputStream</code>	Input stream that allows byte to be returned to the stream

SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after other.
---------------------	--

Table 3.1: The Byte Stream Class

Character Stream Class: Character streams are defined by using two class hierarchies topped by these two abstract classes: Reader and Writer. Reader is used for input, and Writer is used for output. Concrete classes derived from Reader and Writer operates on Unicode character streams. From Reader and Writer are derived several concrete subclasses that handle various I/O situations. In general, the character-based classes parallel the byte-based classes. The character stream classes are shown in Table 3.2

Character Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a chracter array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilerWriter	Filtered writer
InputStreamReader	Input stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output Stream that contains print() and println()
PushbackReader	Input stream that allows character to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after other.

Table 3.2: The Character Stream Class

The Predefined Streams

All Java programs automatically import the java.lang package. This package defines a class called System, which encapsulates several aspects of the run-time environment. Among other things, it contains three predefined stream variables, called in, out, and err. These fields are declared as public, final, and static within System. This means that they can be used by any other part of program and without reference to a specific System object.

System.out refers to the standard output stream. By default, this is the console. System.in refers to standard input, which is by default the keyboard. System.err refers to the standard error stream, which is also the console by default. However, these streams can be redirected to any compatible I/O device. System.in is an object of type InputStream; System.out and System.err are objects of type PrintStream. These are byte streams, even though they are typically used to read and write characters from and to the console. The reason they are byte and not character streams is that the predefined streams were part of the original specification for Java, which did not include the character streams.

Reading Console Input

Originally, the only way to perform console input was to use a byte stream, and much Java code still uses the byte streams exclusively. Today, you can use byte or character streams. For commercial code, the preferred method of reading console input is to use a character-oriented stream. Doing so makes your program easier to internationalize and easier to maintain. It is also more convenient to operate directly on characters rather than converting back and forth between characters and bytes. However, for sample programs, simple utility programs for your own use, and applications that deal with raw keyboard input, using the byte streams is acceptable. For this reason, a console I/O using byte stream is examined here.

Because System.in is an instance of InputStream, you automatically have access to the methods defined by InputStream. Unfortunately, InputStream defines only one input method, read(), which reads bytes. There are three versions of read(), which are shown here:

```
int read( ) throws IOException
int read(byte data[ ]) throws IOException
int read(byte data[ ], int start, int max) throws IOException
```

first version of read() to read a single character from the keyboard (from System.in). It returns -1 when the end of the stream is encountered. The second version reads bytes from the input stream and puts them into data until the array is full, the end of stream is reached, or an error occurs. It returns the number of bytes read, or -1 when the end of the stream is encountered. The third version reads input into data beginning at the location specified by start. Up to max bytes are stored. It returns the number of bytes read, or -1 when the end of the stream is reached. All throw an IOException when an error occurs. When reading from System.in, pressing ENTER generates an end-of-stream condition.

Here is a program that demonstrates reading an array of bytes from System.in. Notice that any I/O exceptions that might be generated are simply thrown out of main(). Such an approach is common when reading from the console.

```
//Read an array of bytes from the keyboard.
```

```
import java. io. *;
class ReadBytes {
    public static void main ( String args[ ])
        throws IOException {
        byte data[ ] = new byte[10];

        System.out.println( "Enter some characters. " );
```

```
        System.in.read (data);
        System.out.println("You entered: ");
        for(int i=0; i< data. length; i++)
            System.out.println( (char) data [i]);
    }
}
```

Here is a sample run:

```
Enter some characters.
Read Bytes
You entered: Read Bytes
```

Reading Characters

Characters can be read from System.in using the read() method defined by BufferedReader in much the same way as they were read using byte streams. There are three versions of read() supported by BufferedReader.

```
int read( ) throws IOException
int read(char data[ ]) throws IOException
int read(char data[ ], int start, int max) throws IOException
```

The first version of read() reads a single Unicode character. It returns -1 when the end of the stream is reached. The second version reads characters from the input stream and puts them into data until either the array is full, the end of file is reached, or an error occurs. It returns the number of characters read or -1 at the end of the stream. The third version reads input into data beginning at the location specified by start. Up to max characters are stored. It returns the number of characters read or -1 when the end of the stream is encountered. All throw an IOException on error. When reading from System.in, pressing ENTER generates an end-of-stream condition.

The following program demonstrates read () by reading characters from the console until the user types a period. Notice that any I/O exceptions that might be generated are simply thrown out of main ().

```
// Use a BufferedReader to read characters from the console.
import java.io.*;
class ReadChars {
    public static void main (String args[ ]) throws IOException
    {
        char c;
        BufferedReader br = new BufferedReader (new InputStreamReader (System. in));
        System.out.println ("Enter characters, period to quit. ");
        // read characters
        do {
            c = (char) br.read ( );
            System.out. println( c );
        } while (c!= ' . ');
    }
}
```

Output:

Enter characters, period to quit.

One Two.

O

n

e

T

w

o

Reading Strings

To read a string from the keyboard, use the version of `readLine( )` that is a member of the `BufferedReader` class. Its general form is shown here:

`String readLine( )` throws `IOException`

It returns a `String` object that contains the characters read. It returns null if an attempt is made to read when at the end of the stream. The following program demonstrates `BufferedReader` and the `readLine( )` method. The program reads and displays lines of text until you enter the word “stop”.

```
//Read a string from console using BufferedReader.
```

```
import java.io.*;
```

```
class ReadLines {
    public static void main (String args [ ]) throws IOException
    {
        // create a BufferedReader using System.in
        BufferedReader br= new BufferedReader (new InputStreamReader (System.in));
        String str;
        System.out.println(“Enter lines of text . “);
        System.out.println(“Enter ‘stop’ to quit. “);
        do {
            str = br.readLine ( ) ;
            System.out.println(str);
        } while (! Str.equals(“stop”));
    }
}
```

Writing console output

As is the case with console input, Java originally provided only byte streams for console output. Java 1.1 added character streams. For the most portable code, character streams are recommended. Because `System.out` is a byte stream, however, byte-based console output is still widely used. Console output is most easily accomplished with `print( )` and `println( )`. These methods are defined by the class `PrintStream` (which is the type of the object referenced by `System.out`). Even though `System.out` is a byte stream, it is still acceptable to use this stream for simple console output. Since `PrintStream` is an output stream derived from `OutputStream`, it also implements the low-level method `write( )`. Thus, it is possible to write to the console by using `write( )`. The simplest form of `write( )` defined by `PrintStream` is shown here:

```
void write(int byteval)
```

This method writes the byte specified by byteval to the file. Although byteval is declared as an integer, only the low-order 8 bits are written. Here is a short example that uses write() to output the character X followed by a new line:

```
//Demonstrate System.out.write ( );
class WriteDemo {
    public static void main (String args [ ]) {
        int b;
        b = 'X';
        System.out.write(b);
        System.out.write ('\n');
    }
}
```

We will not often use write () to perform console output (although it might be useful in some situations), since print() and println() are substantially easier to use. PrintStream supplies two additional output methods: printf() and format(). Both give you detailed control over the precise format of data that you output. For example, you can specify the number of decimal places displayed, a minimum field width, or the format of a negative value.

Short Answer questions (2 marks each)

1. What is stream? List different types of streams in Java.(M.U. Oct/Nov.2014)
2. What is byte and character stream?
3. What is character stream?
4. What are the top two abstract classes of byte stream classes?
5. List out any four byte stream classes with their meaning.
6. What are the top two abstract classes of character stream classes?
7. List out any four character stream classes with their meaning.
8. List the predefined streams of system object
9. What are the three versions of read ()?
10. Write the simplest form of write ()?

Long Answer questions

1. Explain byte stream classes. (5 Marks)
2. Explain any five byte stream classes. (5 Marks)
3. Write a short note on character stream classes. (5Marks)
4. Explain any five character stream classes. (5 Marks)
5. Write a short note on Predefined Streams. (5 Marks)
6. Explain any eight byte stream classes. (8 Marks)
7. Explain any eight character stream classes. (8 Marks)
8. Explain reading console input with an example. (6 Marks)
9. Explain the process of reading characters with an example code. (5 Marks)
10. With syntax and example explain the process of reading a string from the keyboard. (4 marks-M.U.Oct/Nov.2014)

Chapter 6 : Decision Making & Branching

Introduction

A Java program is a set of statements, which are normally executed sequentially in the order in which they appear. This happens when options or repetitions of certain calculations are not necessary. However, in practice, we have a number of situations, where we may have to change the order of execution of statements based on certain conditions, or repeat a group of statements until certain specified conditions are met. This involves a kind of decision making to see whether a particular condition has occurred or not and then direct the computer to execute certain statements accordingly.

When a program breaks the sequential flow and jumps to another part of the code, it is called branching. When the branching is based on a particular condition, it is known as conditional branching. If branching takes place without any decision, it is known as unconditional branching. Java language possesses such decision making capabilities and supports the following statements known as control or decision making statements.

1. if statement
2. switch statement
3. Conditional operator statement

In this Chapter, we shall discuss the features, capabilities and applications of these statements which are classified as selection statements.

Decision Making with If Statement

The if statement is a powerful decision making statement and is used to control the flow of execution of statements. It is basically a no-way decision statement and is used in conjunction with an expression. It takes the following form:



```
if (test expression)
```

It allows the computer to evaluate the expression first and then, depending on whether the value of the expression (relation or condition) is 'true' or 'false', it transfers the control to a particular statement. This point of program has two paths to follow, one for the true condition and the other for the false condition as shown in Fig. 6.1.



Fig. 6.1 Two-way branching

Some examples of decision making, using if statement are:

1. if (bank balance is zero)
 borrow money
2. if (room is dark)
 put on lights
3. if (code is 1)
 person is male
4. if (age is more than 55)
 person is retired

The if statement may be implemented in different forms depending on the complexity of conditions to be tested.

1. Simple if statement
2. if..else statement
3. Nested if..else statement
4. else if ladder

Simple If Statement

The general form of a simple if statement is

```
if(test expression)
{
    statement-block;
}
statement-x;
```

The 'statement-block' may be a single statement or a group of statements. If the test expression is true, the statement-block will be executed; otherwise the statement-block will be skipped and the execution will jump to the statement-x. It should be remembered that when the condition is true both the statement-block and the statement x are executed in sequence. This is illustrated in Fig. 6.2.

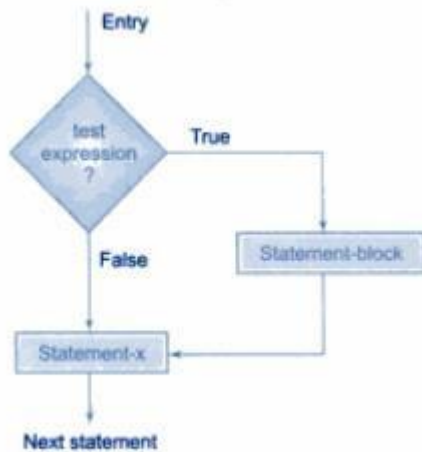


Fig. 6.2 Flowchart of simple if control

Consider the following segment of a program that is written for processing of marks obtained in an entrance examination.

```
if(category == SPORTS)
{
marks = marks + bonus_marks;
}

System.out.println(marks);
```

The program tests the type of category of the student. If the student belongs to the SPORTS category, then additional bonus marks are added to his marks before they are printed. For others, bonus_marks are not added.

Consider a case having two test conditions, one for weight and another for height. This is done using the compound relation

```
if (weight < 50 && height > 170) count = count + 1;
```

This would have been equivalently done using two if statements as follows:

```
if(weight < 50)
if(height > 170) count = count + 1;
```

If the value of weight is less than 50, then the following statement is executed, which in turn is another if statement. This if statement tests height and if the height is greater than 170, then the count is incremented by 1. Program 6.1 illustrates the implementation of the above statement.

Program 6.1 Counting with if statement

```
class IfTest
{
    public static void main(String arg[])
    {
        int i, count, count1, count2;
        float[] weight = { 45.0F, 55.0F, 47.0F, 51.0F, 54.0F };
        float[] height = { 176.5F, 174.2F, 168.0F, 170.7F, 169.0F };
        count = 0;
        count1 = 0;
        count2 = 0;
        for (i = 0; i <= 4; i++)
        {
            if(weight[i] < 50.0 && height[i] > 170.0)
            {
                count1 = count1 + 1;
            }
            count = count + 1; // Total persons
        }
        count2 = count - count1;
        System.out.println("Number of persons with ...");
        System.out.println("Weight<50 and height>170 = " + count1);
        System.out.println("Others = " + count2);
    }
}
```

The output of Program 6.1 will be:

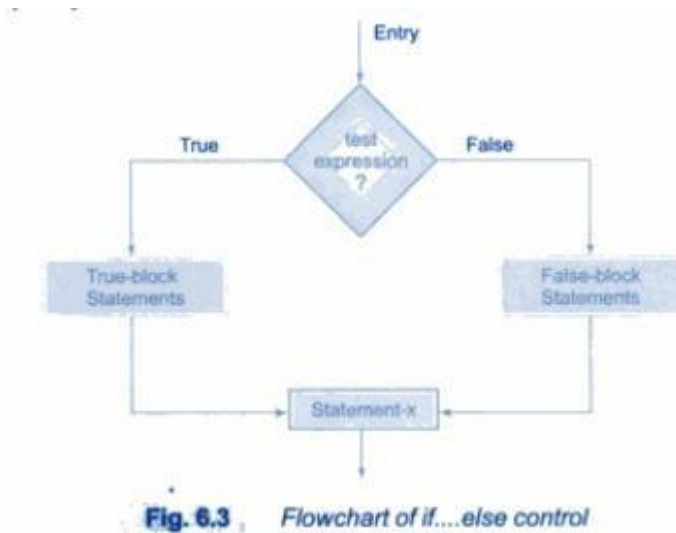
```
Number of persons with ...
Weight < 50 and height > 170 = 1
Others = 4
```

The If...Else Statement

The If...else statement is an extension of the simple if statement. The general form is

```
if(test expression)
{
    True-block statement(s)
}
else
{
    False-block statement(s)
}
statement-x
```

If the test expression is true, then the true-block statement(s) immediately following the if statement, are executed; otherwise, the false-block statement(s) are executed. In either case, either true-block or false-block will be executed, not both. This is illustrated in Fig. 6.3. In both the cases, the control is transferred subsequently to the statement-x.



Let us consider an example of counting the number of boys and girls in a class. We use code 1 for a boy and 2 for a girl. The program statements to do this may be written as follows:

```
if(code== 1)
```

```
    boy = boy + 1;
```

```
if(code==2)
```

```
    girl = girl + 1;
```

```
.....  
.....
```

The first test determines whether or not the student is a boy. If yes, the number of boys is increased by 1 and the program continues to the second test. The second test again determines whether the student is a girl. This is unnecessary. Once a student is identified as a boy, there is no need to test again for a girl. A student can be either a boy or girl, not both. The above program segment can be modified using the else clause as follows:

```
if(code== 1)
```

```
    boy = boy + 1:
```

```
    else
```

```
        girl= girl + 1:
```

Here, if the code is equal to 1, the statement `boy = boy + 1;` is executed and the control is transferred to the statement `xxx`, after skipping the else part. If the code is not equal to 1, the statement `boy=boy + 1;` is skipped and the statement in the else part `girl= girl + 1;` is executed before the control reaches the statement `xxx`.

Program 6.2 counts the even and odd numbers in a list of numbers using the if....else statement. number[] is an array variable containing all the numbers and number.length gives the number of elements in the array.

Program 6.2 Experimenting with if....else statement

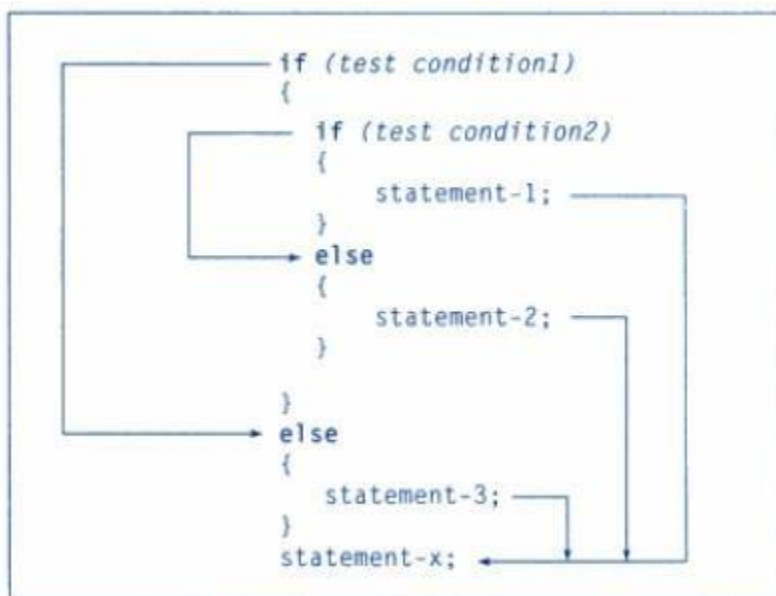
```
class IfElseTest
{
    public static void main(String args[])
    {
        int number[] = { 50, 65, 56, 71, 81 };
        int even = 0, odd = 0;
        for (int i = 0; i < number.length; i++)
        {
            if ((number[i] % 2) == 0) // Decide even or odd
            {
                even += 1; // counting EVEN numbers
            }
            else
            {
                odd += 1; // counting ODD numbers
            }
        }
        System.out.println("Even Numbers : " + even +
            "\nOdd Numbers : " + odd);
    }
}
```

Output of Program 6.2:

Even Numbers : 2 Odd Numbers : 3

Nesting of If.....Else Statements

When a series of decision are involved, we may have to use more than one if..else statements in nested form as follows:



The logic of execution is illustrated in Fig. 6.4. If the condition-1 is false, the statement-3 will be executed; otherwise it continues to perform the second test. If the condition-2 true, the statement-1 will be evaluated; otherwise the statement-2 will be evaluated and then the control is transferred to the statement-x.

A commercial bank has introduced an incentive policy of giving bonus to all its deposit holders. The policy is as follows: A bonus of 2 per cent of the balance held on 31st December is given to every one, irrespective of their balances, and 5 per cent is given to female account holders if their balance is more than Rs 5000. This logic can be coded as follows:

```
if(gender is female)
{
    if (balance > 5000)
        bonus =0.05* balance;
    else
        bonus =0.02* balance;
}
else
{
    bonus -=0.02 balance;
}
balance= balance + bonus;
```

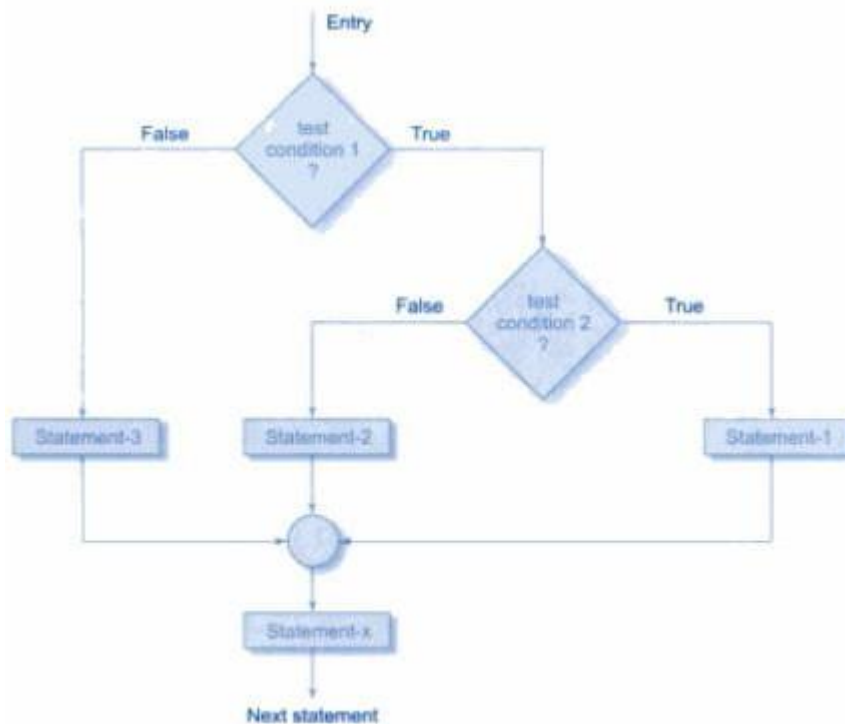


Fig. 6.4 Flowchart of nested if...else statements

When nesting, care should be exercised to match every **if** with an **else**. Consider the following alternative to the above program (which looks right at the first sight):

```
if(gender is female)
    if(balance > 5000)
        bonus 0.05* balance;
    else
        bonus = 0.02 * balance;
balance= balance + bonus;
```

There is an ambiguity as to over which if the else belongs to. In Java an else is linked to the closest non-terminated if. Therefore, the else is associated with the inner if and there is no else option for the outer if. This means that the computer is trying to execute the statement

```
balance = balance + bonus;
```

without really calculating the bonus for the male account holders. Consider another alternative, which also looks correct:

```
if(gender is female)
{
```

```
if (balance > 5000)
    bonus = 0.05 * balance;
}
else
    bonus = 0.02 * balance;
    balance = balance + bonus;
```

In this case, else is associated with the outer if and therefore bonus is calculated for the male account holders. However, bonus for the female account holders, whose balance is equal to or less than 5000 is not calculated because of the missing else option for the inner if.

Program 6.3 employs nested if..else statements to determine the largest of three given numbers.

Program 6.3 Nesting if...else statements

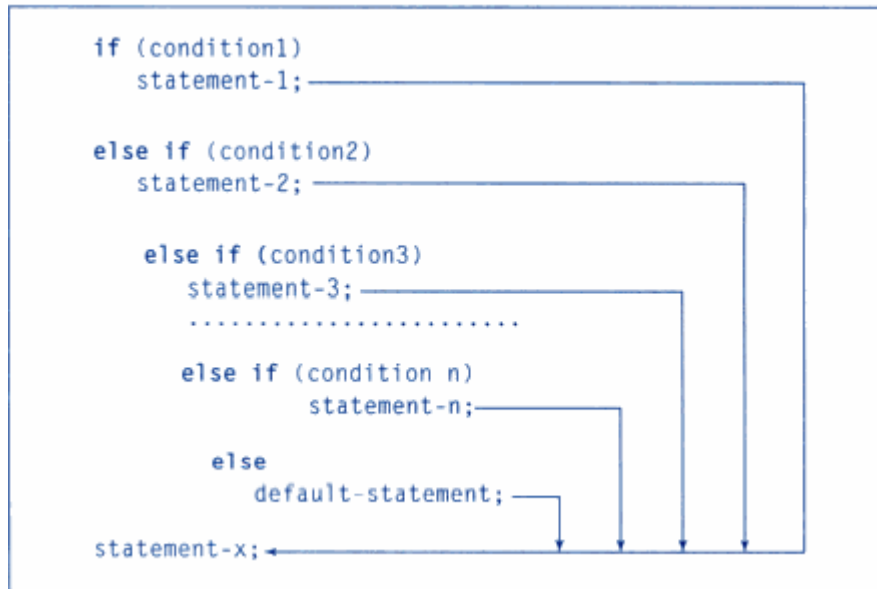
```
class IfElseNesting
{
    public static void main(String args[])
    {
        int a = 325, b = 712, c = 478;
        System.out.print("Largest value is : ");
        if (a > b)
        {
            if (a > c)
            {
                System.out.println(a);
            }
            else
            {
                System.out.println(c);
            }
        }
        else
        {
            if (c > b)
            {
                System.out.println(c);
            }
            else
            {
                System.out.println(b);
            }
        }
    }
}
```

Output of Program 6.3:

Largest value is : 712

The Else If Ladder

There is another way of putting ifs together when multipath decisions are involved. A multipath decision is a chain of ifs in which the statement associated with each else is an if. It takes the following general form:



This construct is known as the else if ladder. The conditions are evaluated from the top (of the ladder), downwards. As soon as the true condition is found, the statement associated with it is executed and the control is transferred to the statement-x (skipping the rest of the ladder). When all the n conditions become false, then the final else containing the default-statement will be executed. Figure 6.5 shows the logic of execution of else if ladder statements.

Let us consider an example of grading the students in an academic institution. The grading is done according to the following rules:

Average marks	Grade
80 to 100	Honours
60 to 79	First Divisions
50 to 59	Second Division
40 to 49	Third Divisions
0 to 39	Fail

This grading can be done using the else if ladder as follows:

```
if(marks > 79)
    grade = "Honours";
else if(marks > 59)
    grade = "First Division";
else if(marks > 49)
    grade = "Second Division";
else if(marks > 39)
    grade = "Third Division";
else
    grade = "Fail";
System.out.println("Grade: +grade);
```

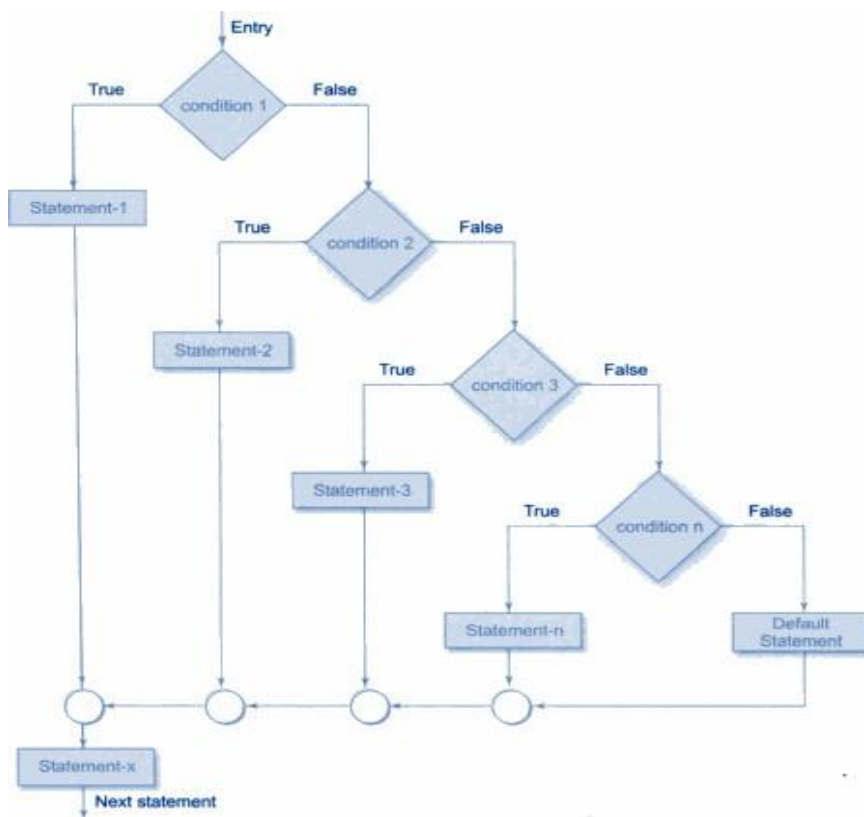


Fig. 6.5 Flowchart of else...if ladder

Consider another example given below:

```
.....
if (code == 1)
    colour = "RED";
else if (code == 2)
    colour = "GREEN";
    else if (code == 3)
        code = "WHITE";
    else
        colour = "Yellow";
.....
```

Code numbers other than 1, 2 or 3 are considered to represent YELLOW colour. The same results can be obtained by using nested if. else statements.

```
if(code != 1)
    if (code != 2)
        if (code != 3)
            colour = "YELLOW";
        else
            colour = "WHITE";
    else
        colour = "GREEN";
else
    colour = "RED";
```

In such situations, the choice of the method is left to the programmer. However, in order to choose an if structure that is both effective and efficient, it is important that the programmer is fully aware of the various forms of an if statement and the rules governing their nesting. Program 6.4 demonstrates the use of if. else ladder in analysing a mark list.

Program 6.4 Demonstration of else if ladder

```
class ElseIfLadder
{
    public static void main(String args[])
    {
        int rollNumber[] = { 111, 222, 333, 444 };
        int marks[] = { 81, 75, 43, 58 };
        for (int i = 0; i < rollNumber.length; i++)
        {
            if (marks[i] > 79)
                System.out.println(rollNumber[i] + " Honours");
            else if (marks[i] > 59)
```

(Continued)

Program 6.4 (Continued)

```
                System.out.println(rollNumber[i] + " I Division");
            else if (marks[i] > 49)
                System.out.println(rollNumber[i] + " II Division");
            else
                System.out.println(rollNumber[i] + " FAIL");
        }
    }
}
```

Program 6.4 produces the following output:

```
111 Honours
222 I Division
333 FAIL
444 II Division
```

The Switch Statement

We have seen that when one of the many alternatives is to be selected, we can design a program using if statements to control the selection. However, the complexity of such a program increases dramatically when the number of alternatives increases. The program becomes difficult to read and follow. At times, it may confuse even the designer of the program. Fortunately, Java has a built-in multiway decision statement known as switch. The switch statement tests the value of a given variable (or expression) against a list of case values and when a match is found, a block of statements associated with that case is executed. The general form of the switch statement is as shown below:

```
switch (expression)
{
    case value-1:
        block-1
        break;
    case value-2:
        block-2
        break;
    .....
    default:
        default-block
        break;
}
statement-x;
```

The expression is an integer expression or characters. value-1, value-2.... are constants or constant expressions (evaluable to an integral constant) and are known as case labels. Each of these values should be unique within a switch statement. block-1, block-2.... are statement lists and may contain zero or more statements. There is no need to put braces around these blocks but it is important to note that case labels end with a colon (:).

When the switch is executed, the value of the expression is successively compared against the values value-1, value-2,. If a case is found whose value matches with the value of the expression, then the block of statements that follows the case are executed.

The break statement at the end of each block signals the end of a particular case and causes an exit from the switch statement, transferring the control to the statement-x following the switch.

The default is an optional case. When present, it will be executed if the value of the expression does not match with any of the case values. If not present, no action takes place when all matches fail and the control goes to the statement-x.

The selection process of switch statement is illustrated in the flowchart shown in Fig. 6.6.

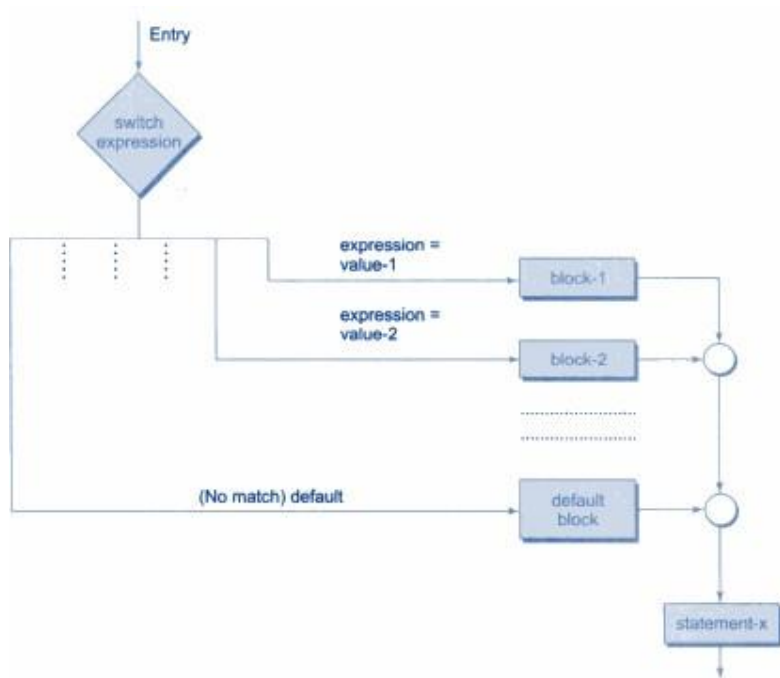


Fig. 6.6 Selection process of the switch statement

The switch statement can be used to grade the students as discussed in the last section. This is illustrated below:

```
index= marks/10;
```

```
switch(index)
```

```
{
```

```
    case 10:
```

```
    case 9:
```

```
    case 8:
```

```
        grade = "Honours";
```

```
        break;
```

```
    case 7:
```

```
    case 6:
```

```
        grade= "First Division";
```

```
        break;
```

```
    case 5:
```

```
        grade = "Second Division";
```

```
        break;
```

case 4:

```
grade= "Third Division";
```

```
break;
```

default:

```
grade= "Fail";
```

```
break;
```

```
}
```

```
System.out.println(grade);
```

Note that we have used a conversion statement

```
index = marks/10;
```

where, index is defined as an integer. The variable index takes the following integer values.

Marks	Index
100	10
90-99	9
80-89	8
70-79	7
60-69	6
50-59	5
40-49	4
30-39	3
20-29	2
10-19	1
0-9	0

The segment of the program illustrates two important features. First, it uses empty cases. The first three cases will execute the same statements

```
grade = "Honours"
```

```
break;
```

Same is the case with case 7 and case 6. Second, default condition is used for all other cases where marks is less than 40.

Program 6.5 illustrates the use of **switch** for designing a menu driven interactive program.

Program 6.5 Testing the switch ()

```
class CityGuide
{
    public static void main(String args[])
    {
        char choice;
        System.out.println("Select your choice");
        System.out.println(" M -> Madras");
        System.out.println(" B -> Bombay");
        System.out.println(" C -> Calcutta");
        System.out.print("Choice ---->");
        System.out.flush();
        try
        {
            switch (choice = (char) System.in.read())
            {
                case 'M':
                case 'm': System.out.println("Madras : Booklet 5");
                           break;
                case 'B':
                case 'b': System.out.println("Bombay : Booklet 9");
                           break;
                case 'C':
                case 'c': System.out.println("Calcutta: Booklet15");
                           break;
                default: System.out.println("Invalid Choice (IC)");
            }
        }
        catch (Exception e)
        {
            System.out.println("I/O Error");
        }
    }
}
```

Output of the Program 6.5:

```
Run1
Select your choice
 M ----> Madras
 B ----> Bombay
 C ----> Calcutta
Choice ----> m
Madras : Booklet 5
```

```
Run2
Select your choice
 M ----> Madras
 B ----> Bombay
 C ----> Calcutta
Choice ----> M
Madras : Booklet 5
```

```
Run3
Select your choice
 M ----> Madras
 B ----> Bombay
 C ----> Calcutta
Choice ----> c
Calcutta : Booklet15
```


UNIT 2

Chapter 1 - Decision making & Looping

Introduction

A computer is well suited to perform repetitive operations. It can do it tirelessly for 10, 100 or even 10,000 times. Every computer language must have features that instruct a computer to perform such repetitive tasks. The process of repeatedly executing a block of statements is known as looping. The statements in the block may be executed any number of times, from zero to infinite number. If a loop continues forever, it is called an infinite loop.

Java supports such looping features which enable us to develop concise programs containing repetitive processes without using unconditional branching statements like goto statement.

In looping, a sequence of statements are executed until some conditions for the termination of the loop are satisfied. A program loop therefore consists of two segments, one known as the body of the loop and the other known as the control statement. The control statement tests certain conditions and then directs the repeated execution of the statements contained in the body of the loop.

Depending on the position of the control statement in the loop, a control structure may be classified either as the entry-controlled loop or as exit-controlled loop. The flowcharts in Fig. 7.1 illustrate these structures. In the entry-controlled loop, the control conditions are tested before the start of the loop execution. If the conditions are not satisfied, then the body of the loop will not be executed. In the case of an exit-controlled loop, the test is performed at the end of the body of the loop and therefore the body is executed unconditionally for the first time.

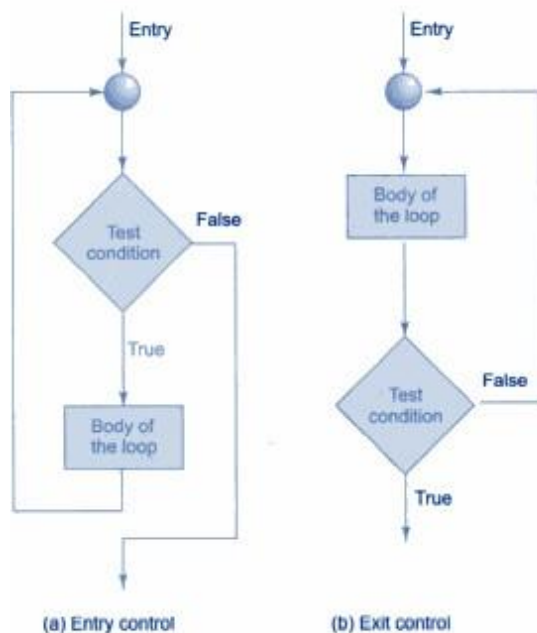


Fig. 7.1 Loop control structures

The test conditions should be carefully stated in order to perform the desired number of loop executions. It is assumed that the test condition will eventually transfer the control out of the loop. In case, due to some reason it does not do so, the control sets up an infinite loop and the body is executed over and over again.

A looping process, in general, would include the following four steps:

1. Setting and initialization of a counter.
2. Execution of the statements in the loop.
3. Test for a specified condition for execution of the loop.
4. Incrementing the counter.

The test may be either to determine whether the loop has been repeated the specified number of times or to determine whether a particular condition has been met with.

The Java language provides for three constructs for performing loop operations. They are:

1. while construct
2. do construct
3. for construct

We shall discuss the features and applications of each of these constructs in this chapter.

The While Statement

The simplest of all the looping structures in Java is the while statement. The basic format of the while statement is

```
Initialization:
While (test condition)
{
    Body of the loop
}
```

The while is an entry-controlled loop statement. The test condition is evaluated and if the condition is true, then the body of the loop is executed. After execution of the body, the test condition is once again evaluated and if it is true, the body is executed once again. This process of repeated execution of the body continues until the test condition finally becomes false and the control is transferred out of the loop. On exit, the program continues with the statement immediately after the body of the loop.

The body of the loop may have one or more statements. The braces are needed only if the body contains two or more statements. However, it is a good practice to use braces even if the body has only one statement.

Consider the following code segment:

```
sum = 0;
n = 1;
while(n <= 10)
{
    sum = sum + n * n;
    n = n+1;
}
System.out.println("Sum = "+ sum);
.....
```

The body of the loop is executed 10 times for n = 1, 2, 10 each time adding the square of the value of n, which is incremented inside the loop.

The test condition may also be written as n < 11; the result would be the same

Program 7.1 Using while loop

```
class WhileTest
{
    public static void main(String args[ ])
    {
```

(Continued)

Program 7.1 (Continued)

```
        StringBuffer string = new StringBuffer();
        char c;
        System.out.println("Enter a string ");
        try
        {
            while ( (c = (char)System.in.read()) != '\n')
            {
                string.append(c); // Append character
            }
        }
        catch (Exception e)
        {
            System.out.println("Error in input");
        }
        System.out.println(" You have entered .... ");
        system.out.println(string);
    }
}
```

Given below is the output of Program 7.1:

```
Enter a string
Java is a true Object-Oriented Language
You have entered ...
Java is a true Object-Oriented Language
```

The do statement

The while loop construct that we have discussed in the previous section makes a test condition before the loop is executed. Therefore, the body of the loop may not be executed at all if the condition is not satisfied at the very first attempt. On some occasions it might be necessary to execute the body of the loop before the test is performed. Such situations can be handled with the help of the do statement. This takes the form:

```
Initialization;  
do  
{  
    Body of the loop  
}  
while (test condition);
```

On reaching the do statement, the program proceeds to evaluate the body of the loop first. At the end of the loop, the test condition in the while statement is evaluated. If the condition is true, the program continues to evaluate the body of the loop once again. This process continues as long as the condition is true. When the condition becomes false, the loop will be terminated and the control goes to the statement that appears immediately after the while statement.

Since the test condition is evaluated at the bottom of the loop, the do...while construct provides an exit-controlled loop and therefore the body of the loop is always executed at least once. Consider an example:

```
i = 1;  
sum = 0;  
do  
{  
    sum = sum + i;  
    i = i+2;  
}  
while(sum < 40 || i < 10);  
.....  
.....
```

The loop will be executed as long as one of the two relations is true. Program 7.2 illustrates the use of do...while loops for printing a multiplication table.

Program 7.2 Printing multiplication table using do....while loop

```
class DowhileTest  
{  
    public static void main(String args[])  
    {  
        int row, column, y;  
        System.out.println("Multiplication Table \n");  
        row = 1;  
        do  
        {  
            column = 1;  
            do  
            {  
                y = row * column;  
                System.out.print(" " + y);  
                column = column + 1;  
            }  
            while (column <= 3);  
            System.out.println("\n");  
            row = row + 1;  
        }  
        while (row <= 3);  
    }  
}
```

Program 7.2 uses two **do-while** loops in nested form and produces the following output:

Multiplication Table

1	2	3
2	4	6
3	6	9

The for Statement

The for loop is another entry-controlled loop that provides a more concise loop control structure. The general form of the for loop is

```
for (initialization ; test condition ; increment)
{
    Body of the loop
}
```

The execution of the for statement is as follows:

1. Initialization of the control variables is done first, using assignment statements such as `i=1` and `count=0`. The variables `i` and `count` are known as loop-control variables.
2. The value of the control variable is tested using the test condition. The test condition is a relational expression, such as `i < 10` that determines when the loop will exit. If the condition is true, the body of the loop is executed; otherwise the loop is terminated and the execution continues with the statement that immediately follows the loop.
3. When the body of the loop is executed, the control is transferred back to the for statement after evaluating the last statement in the loop. Now, the control variable is incremented using an assignment statement such as `i=i+1` and the new value of the control variable is again tested to see whether it satisfies the loop condition. If the condition is satisfied, the body of the loop is again executed. This process continues till the value of the control variable fails to satisfy the test condition.

Consider the following segment of a program

```
for (x = 0 ; x <= 9; x = x+1)
{
    System.out.println(x);
}
```

This for loop is executed 10 times and prints the digits 0 to 9 in one line. The three sections enclosed within parentheses must be separated by semicolons. Note that there is no semicolon at the end of the increment section, `x=x+1`.

The for statement allows for negative increments. For example, the loop discussed above can be written as follows:

```
for (x = 9; x >= 0; x = x-1)
    System.out.println(x);
```

This loop is also executed 10 times, but the output would be from 9 to 0 instead of 0 to 9. Note that braces are optional when the body of the loop contains only one statement.

Since the conditional test is always performed at the beginning of the loop, the body of the loop may not be executed at all, if the condition fails at the start, For example,

```
for (x = 9; x < 9; x = x-1)
{
    .....
    .....
}
```

will never be executed because the test condition fails at the very beginning itself.

Let us consider the problem of sum of squares of integers discussed in Section 7.2. This problem can be coded using the for statement as follows:

```
.....
sum = 0;
for (n = 1; n <= 10; n = n+1)
{
    sum = sum + n*n;
}
.....
.....
```

The body of the loop

Sum= Sum + n*n:

is executed 10 times for n= 1, 2, ..., 10 each time incrementing the sum by the square of the value of n. One of the important points about the for loop is that all the three actions, namely initialization, testing and incrementing, are placed in the for statement itself, thus making them visible to the programmes and users, in one place. The for statement and its equivalent of while and do statements are shown in Table 7.1.

Table 7.1 Comparison of the Three Loops		
for	while	do
for (n=1;n<=10;++n) { }	n = 1 while (n<=10) { n = n+1; }	n = 1 do { n = n+1 } while (n<=10);

Program 7.3 illustrates the use of for loop for computing and printing the power of 2 table.

Program 7.3 Computing the 'power of 2' using for loop

```
Class ForTest
{
    public static void main(String args[ ])
    {
        long p;
        int n;
        double q;
        System.out.println("2 to power -n    n    2 to power n");

        p = 1;
        for (n = 0; n < 10; ++n)
        {
            if (n == 0)
                p = 1;
            else
                p = p * 2;
            q = 1.0 / (double)p;
            System.out.println(" " + q + " " + n + " " + p);
        }
    }
}
```

Output of Program 7.3 would be:

2 to power -n	n	2 to power n
1	0	1
0.5	1	2
0.25	2	4
0.125	3	8
0.0625	4	16
0.03125	5	32
0.015625	6	64
0.0078125	7	128
0.00390625	8	256
0.00195313	9	512

Additional Features of for Loop

The for loop has several capabilities that are not found in other loop constructs. For example, more than one variable can be initialized at a time in the for statement. The statements

```
p = 1;
for (n=0; n<17; ++n)
```

Can be written as

```
for (p=1, n=0; n<17; ++n)"
```

Notice that the initialization section has two parts $p=1$ and $n=1$ separated by a comma.

Like the initialization section, the increment section may also have more than one part. For example, the loop

```
for (n=1, m=50; n<=m; n=n+1, m=m-1)
{
    .....
    .....
}
```


is perfectly valid. The multiple arguments in the increment section are separated by commas. The third feature is that the test condition may have any compound relation and the testing need not be limited only to the loop control variable. Consider the example that follows:

```
sum = 0;
for (i = 1, i < 20 && sum < 100; ++i)
{
    .....
    .....
}
```

The loop uses a compound test condition with the control variable *i* and external variable *sum*. The loop is executed as long as both the conditions *i* < 20 and *sum* < 100 are true. The *sum* is evaluated inside the loop.

It is also permissible to use expressions in the assignment statements of initialization and increment sections. For example, a statement of the type

```
for (x = (m+n)/2; x > 0; x = x/2)
```

is perfectly valid.

Another unique aspect of for loop is that one or more sections can be omitted, if necessary. Consider the following statements:

```
.....
m = 5;
for ( ; m != 100 ; )
{
    System.out.println(m);
    m = m+5;
}
.....
```

Both the initialization and increment sections are omitted in the for statement. The initialization has been done before the for statement and the control variable is incremented inside the loop. In such cases, the sections are left blank. However, the semicolons separating the sections must remain. If the test condition is not present, the for statement sets up an infinite loop.

We can set up time delay loops using the null statement as follows:

```
for (j = 1000; j > 0; j = j-1)
;
```

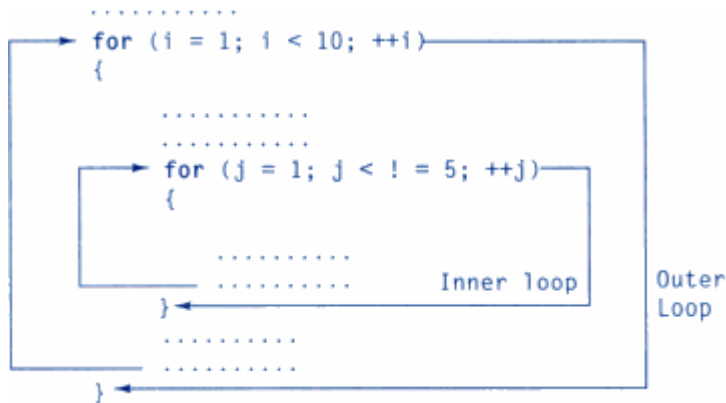
This loop is executed 1000 times without producing any output; it simply causes a time delay. Notice that the body of the loop contains only a semicolon, known as a empty statement. This can also be written as

```
.....
for (j=1000; j > 0; j = j-1);
.....
```

This implies that the compiler will not give an error message if we place a semicolon by mistake at the end of a for statement. The semicolon will be considered as an empty statement and the program may produce some nonsense.

Nesting of for Loops

Nesting of loops, that is, one for statement within another for statement, is allowed in Java. We have used this concept in Program 7.2. Similarly, for loops can be nested as follows:



The loops should be properly indented so as to enable the reader to easily determine which statements are contained within each for statement.

A program segment to print a multiplication table using for loops is shown below:

```
.....
for (row = 1; row <= ROWMAX; ++row)
{
    for (column = 1; column <= COLMAX; ++column)
    {
        Y = row * column;
        System.out.print(" " + Y);
    }
    System.out.println(" ");
}
.....
```

The outer loop controls the rows while the inner one controls the columns.

The Enhanced for Loop

The enhanced for loop, also called for each loop, is an extended language feature introduced with the J2SE 5.0 release. This feature helps us to retrieve the array of elements efficiently rather than using array indexes. We can also use this feature to eliminate the iterators in a for loop and to retrieve the elements from a collection. The enhanced for loop takes the following form:

```
for (Type Identifier : Expression)
{
    //statements;
}
```

where, Type represents the data type or object used; Identifier refers to the name of a variable; and Expression is an instance of the `java.lang.Iterable` interface or an array. For example, consider the following statements:

```
int numarray[3] = {56, 48, 79};
for (int k=0; k<=3; k++)
{
    if (numarray[k]>50 && numarray[k]<100)
    {
        System.out.println("The selected value is "+numarray[k]);
    }
}
```

which is equivalent to the following code:

```
int numarray[3] = {56, 48, 79};
for (int k:numarray)
{
    if (k>50 && k<100)
    {
        System.out.println("The selected value is "+k);
    }
}
```

Thus, we can use the enhanced for loop to track the elements of an array efficiently. In the same manner, we can track the collection elements using the enhanced for loop as follows:

```
Stack samplestack = new Stack();
samplestack.push(new Integer(56));
samplestack.push(new Integer(48));
samplestack.push(new Integer(79));
for(Object obj : samplestack)
{
    System.out.println(obj);
}
```

Jumps in loops

Loops perform a set of operations repeatedly until the control variable fails to satisfy the test condition. The number of times a loop is repeated is decided in advance and the test condition is written to achieve this. Sometimes, when executing a loop it becomes desirable to skip a part of the loop or to leave the loop as soon as a certain condition occurs. For example, consider the case of searching for a particular name in a list containing, say, 100 names. A program loop written for reading and testing the names a 100 times must be terminated as soon as the desired name is found. Java permits a jump from one statement to the end or beginning of a loop as well as a jump out of a loop.

Jumping Out of a Loop

An early exit from a loop can be accomplished by using the `break` statement. We have already seen the use of the `break` in the switch statement. This statement can also be used within while, do or for loops as illustrated in Fig. 7.2.

When the `break` statement is encountered inside a loop, the loop is immediately exited and the program continues with the statement immediately following the loop. When the loops are nested, the `break` would only exit from the loop containing it. That is, the `break` will exit only a single loop.

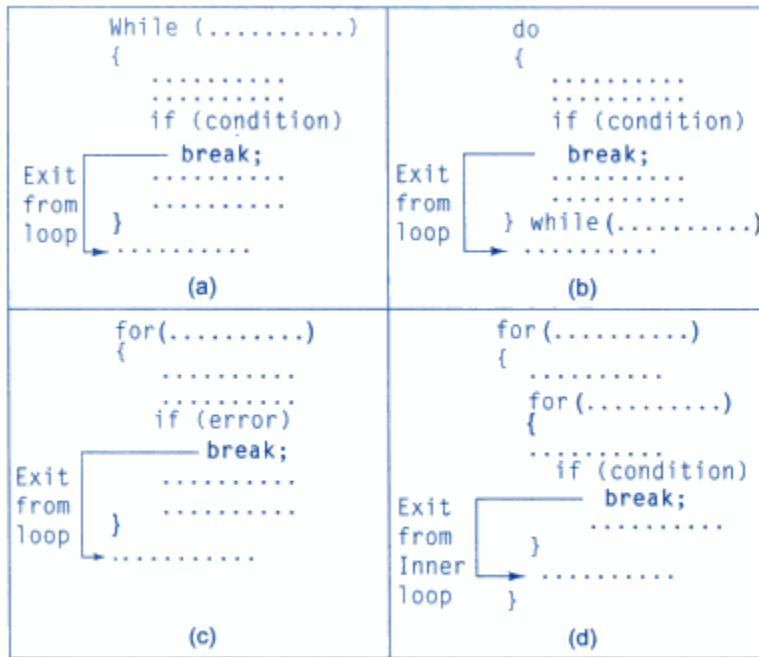


Fig. 7.2 Exiting a loop with `break` statement

Skipping a Part of a Loop

During the loop operations, it may be necessary to skip a part of the body of the loop under certain conditions. For example, in processing of applications for some job, we might like to exclude the processing of data of applicants belonging to a certain category. On reading the category code of an applicant, a test is made to see whether his application should be considered or not. If it is not to be considered, the part of the program loop that processes the application details is skipped and the execution continues with the next loop operation.

Like the `break` statement, Java supports another similar statement called the `continue` statement. However, unlike the `break` which causes the loop to be terminated, the `continue`, as the name implies, causes the loop to be continued with the next iteration after skipping any statements in between. The `continue` statement tells the compiler, "SKIP THE FOLLOWING STATEMENTS AND CONTINUE WITH THE NEXT ITERATION". The format of the `continue` statement is simply

Continue;

The use of the `continue` statement in loops is illustrated in Fig. 7.3. In `while` and `do` loops, `continue` causes the control to go directly to the test condition and then to continue the iteration process. In the case of `for` loop, the increment section of the loop is executed before the test condition is evaluated.

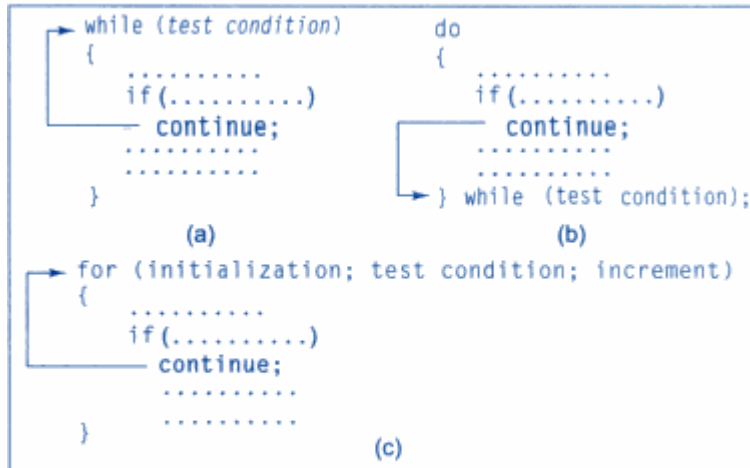


Fig. 7.3 *Bypassing and continuing in loops*

Labelled loops

In Java, we can give a label to a block of statements. A label is any valid Java variable name. To give a label to a loop, place it before the loop with a colon at the end. Example:

```
loop1:   for (.....)
{
    .....
}
.....
```

A block of statements can be labelled as shown below:

```
block1:  {
    .....
    block2: {
        .....
    }
    .....
}
```

We have seen that a simple `break` statement causes the control to jump outside the nearest loop and a simple `continue` statement restarts the current loop. If we want to jump outside a nested loops or to continue a loop that is outside the current one, then we may have to use the labelled `break` and labelled `continue` statements. Example:

```
outer: for (int m = 1; m<11; m++)
{
    for (int n = 1; n<11; n++)
    {
        System.out.print(" " + m*n);
        if (n == m)
            continue outer;
    }
}
```

Here, the continue statement terminates the inner loop when $n = m$ and continues with the next iteration of the outer loop (counting m).

Another example:

```
loop1: for (int i = 0; i < 10; i++)
{
    loop2: while (x < 100)
    {
        Y = i * x;
        if (Y > 500)
            break loop1;
        .....
    }
    .....
}
```

Jumping out of both loops

Here, the label loop1 labels the outer loop and therefore the statement

break loop1;

causes the execution to break out of both the loops. Program 7.4 illustrates the use of break and continue statements.

Program 7.5 Use of continue and break statements

```
class ContinueBreak
{
    public static void main(String args[])
    {
        LOOP1 : for (int i = 1; i < 100; i++)
        {
            System.out.println(" ");
            if (i >= 10) break;
            for (int j = 1; j < 100; j++)
            {
                System.out.print(" * ");
            }
        }
    }
}
```

(Continued)

Program 7.5 (Continued)

```
        if (j == 1)
            continue LOOP1;
    }
    System.out.println("Termination by BREAK");
}
```

Program 7.5 produces the following output:

```
*
*  *
*  *  *
*  *  *  *
*  *  *  *  *
*  *  *  *  *  *
*  *  *  *  *  *  *
*  *  *  *  *  *  *  *
*  *  *  *  *  *  *  *  *
Termination by BREAK
```


Chapter2 : Class & Objects

Class Fundamentals:

Java is a true object-oriented language and therefore the underlying structure of all Java programs is classes. Anything we wish to represent in a Java program must be encapsulated in a class that defines the *state* and *behaviour* of the basic program components known as *objects*.

Classes provide a convenient method for packing together a group of logically related data items and functions that work on them. In Java, the data items are called fields and the functions are called *methods*. Calling a specific method in an object is described as sending the object a message.

A class is essentially a description of how to make an object that contains fields and methods. It provides a sort of *template* for an object and behaves like a basic data type such as `int` it is therefore important to understand how the fields and methods are defined in a class and how they are used to build a Java program that incorporates the basic OOP concepts such as *encapsulation*, *inheritance* and *polymorphism*.

General Form of a class: A class is a user-defined data type with a template that serves to define its properties. Once the class type has been defined, we can create "variables" of that type. In Java, these variables are termed as *instances* of classes, which are the actual *objects*. The basic form of a class definition is:

```
class    classname [extends superclassname]
{
    [ variable declaration; ]
    [ methods declaration; ]
}
```

Everything inside the square brackets is optional. Even body of the class can be empty. Because of the body is empty, this class does not contain any properties and therefore cannot do anything. We can, however, compile it and even create objects using it. Classname and superclassname are any valid Java identifiers. The keyword `extends` indicates that the properties of the superclassname class are extended to the classname class. This concept is known as inheritance.

Data is encapsulated in a class by placing data fields inside the body of the class definition. These variables are called instance variables because they are created whenever an object of the class is instantiated. We can declare the instance variables exactly the same way as we declare local variables. Example:

```
class Rectangle
{
    int width;
    int length;
}
```

The class **Rectangle** contains two integer type instance variables. It is allowed to declare them in one line as

```
int length, width;
```

These variables are only declared and therefore no storage space has been created in the memory. Instance variables are also known as member variables.

Creating objects

An object in Java is essentially a block of memory that contains space to store all the instance variables. Creating an object is also referred to as instantiating an object.

Objects in Java are created using the new operator. The new operator creates an object of the specified class and returns a reference to that object. Here is an example of creating an object of type Rectangle.

```
Rectangle rect1 // declare
rect1 = new Rectangle ( ) // instantiate
```

The first statement declares a variable to hold the object reference and the second one actually assigns the object reference to the variable. The variable rect1 is now an object of the Rectangle class. Both statements can be combined into one as shown below:

```
Rectangle rect1 = new Rectangle ( );
```

The method Rectangle () is the default constructor of the class. We can create any number of objects of Rectangle.

```
Rectangle rect1 = new Rectangle( );
Rectangle rect2 = new Rectangle( );
```

It is important to understand that each object has its own copy of the instance variables of its class. This means that any changes to the variables of one object have no effect on the variables of another.

Assigning Object Reference Variables

In an assignment operation, object reference variables act differently than do variables of a primitive type, such as int. When you assign one primitive-type variable to another, the situation is straightforward. The variable on the left receives a copy of the value of the variable on the right. When you assign one object reference variable to another, the situation is a bit more complicated because you are changing the object that the reference variable refers to. The effect of this difference can cause some counterintuitive results. For example, consider the following fragment

```
Vehicle car1= new Vehicle ( );
Vehicle car2= car1
```

car1 and car2 will both refer to the same object. The assignment of car1 to car2 simply makes car2 refer to the same object as does car1. Thus, the object can be acted upon by either car1 or car2. Although car1 and car2 both refer to the same object, they are not linked in any other way.

Introducing Methods

A class with only data fields (and without methods that operate on that data) has no life. The objects created by such a class cannot respond to, any, messages. We must therefore add methods that are necessary for manipulating the data contained in the class. The general form of a method declaration is

```
type methodname (parameter-list)
{
    method-body;
}
```

Method declarations have four basic parts:

- The name of the method (method name)
- The type of the value the method returns (type)
- A list of parameters (parameter-list)
- The body of the method

The type specifies the type of value the method would return. This could be a simple data type such as int as well as any class type. It could even be void type, if the method does not return any value. The method name is a valid identifier. The parameter list is always enclosed in parentheses. This list contains variable names and types of all the values we want to give to the method as input. The variables in the list are separated by commas. In the case where no input data are required, the declaration must retain the empty parentheses.

Examples:

```
(int m, float x, float y) // Three parameters
( ) // Empty list
```

The body actually describes the operations to be performed on the data. Let us consider the Rectangle class again and add a method getData () to it.

```
class Rectangle
{
    int length;
    int width;
    void getData(int x , int y )
    {
        length= x ;
        width= y ;
    }
}
```

Note that the method has a return type of void because it does not return any value. We pass two integer values to the methods which are then assigned to the instance variables length and width. The getData() method is basically added to provide values to the instance variables. Let us add some more properties to the class. Assume that we want to compute the area of the rectangle defined by the class. This can be done as follows:

```
class Rectangle
{
    int length, width; // Com bind declaration
    void getData(int x , int y)
    {
        length = x ;
        width = Y ;
    }
    int rectArea( )
    {
        int area = length * width;
        return (area)
    }
}
```

The new method `rectArea()` computes area of the rectangle and returns the result. Since the result would be an integer, the return type of the method has been specified as `int`. Also note that the parameter list is empty. Declaration of instance variables (and also local variables) can be combined as

```
int length, width;
```

The parameter list used in the method header should always be declared independently separated by commas. That is,

```
void getData (int x, y) // Incorrect is illegal.
```

Now, our class `Rectangle` contains two instance variables and two methods. We can add more variables and methods, if necessary. Instance variables and methods in classes are accessible by all the methods in the class but a method cannot access the variables declared in other methods.

Example:

```
class Access
{
    int x ;
    void method1( )
    {
        int y;
        x = 10 ; // legal
        y = x    // legal
    }
    void method2( )
    {
        int z ;
        x = 5;  //legal
        z = 10; //legal
        y = 1;  //illegal
    }
}
```

Returning From a Method

In general, there are two conditions that cause a method to return, when the method's closing curly brace is encountered. The second is when a return statement is executed. There are two forms of return, one for use in void methods (those that do not return a value) and one for returning values. In a void method, you can cause the immediate termination of a method by using this form of return:

```
return;
```

When this statement executes, program control returns to the caller, skipping any remaining code in the method. For example, consider this method:

```
void myMeth ( ) {
    int i;
    for (i=0; i<10; i++) {
        if ( i == 5) return;  // stop at 5
        System.out.println( );
    }
}
```

Here, the for loop will only run from 0 to 5, because once `i` equals 5, the method returns. It is permissible to have multiple return statements in a method, especially when there are two or more routes out of it.

Returning a Value

Although methods with a return type of void are not rare, most methods will return a value. In fact, the ability to return a value is one of the most useful features of a method. Return values are used for a variety of purposes in programming. In some cases, such as with `sqrt( )`, the return value contains the outcome of some calculation. In other cases, the return value may simply indicate success or failure. In still others, it may contain a status code. Methods return a value to the calling routine using this form of return:

return value;

Here, value is the value returned. This form of return can be used only with methods that have a non-void return type.

Example:

```
class Rectangle
{
    int length, width;           // Declaration of variables
    void getData(int x, int y)   // Definition of method
    {
        length = x;
        width = y;
        int rectArea ( )        // Definition of another method
        {
            int area = length * width;
            return (area);
        }
    }
}

class RectArea                  // Class with main method
{
    public static void main (String args[ ])
    {
        int areal,area2;
        Rectangle rect1 = new Rectangle(); //Creating objects
        Rectangle rect2 = new Rectangle();
        rect1.length = 15;                // Accessing variables
        rect1. width = 10;
        areal = rect1.length * rect1.width; // Accessing methods
        rect2.getData(20,12);
        area2 = rect2.rectArea();
        System.out.println("Area1 = "+ areal);
        System.out.println("Area2 = "+ area2);
    }
}
```

Output:

```
Area1= 150
Area1=240
```

Using Parameters

It is possible to pass one or more values to a method when the method is called. Inside the method, the variable that receives the argument is called a parameter. Parameters are declared

inside the parentheses that follow the method's name. The parameter declaration syntax is the same as that used for variables. A parameter is within the scope of its method, and aside from its special task of receiving an argument, it acts like any other local variable.

Example:

```
class Rectangle
{
    int length, width;           // Declaration of variables
    void getData(int x, int y)   // Definition of method
    {
        length = x;
        width = y;
    }
}

class RectArea                  // Class with main method
{
    public static void main (String args[ ])
    {
        Rectangle rect1 = new Rectangle(); //Creating objects
        rect1.length = 15;                // Accessing variables
        rect1. width = 10;
        System.out.println("Length = "+ length);
        System.out.println("Area2 = "+ width);
    }
}
```

Constructors

All objects that are created must be given initial values. We can do these using two approaches. The first approach uses the dot operator to access the instance variables and then assigns values to them individually. It can be a tedious approach to initialize all the variables of all the objects. The second approach takes the help of a method like `getData` to initialize each object individually using statements like,

```
rect1.getData (15, 10);
```

It would be simpler and more concise to initialize an object when it is first created. Java supports a special type of method, called a constructor that enables an object to initialize itself when it is created. Constructors have the same name as the class itself. They do not return any value, not even void. This is because they return the instance of the class itself.

Let us consider our `Rectangle` class again. We can now replace the `getData` method by a constructor method as shown below:

```
class Rectangle
{
    int length;
    int width;
    Rectangle ( ) // Constructor method
    {
        length = 10 ;
    }
}
```

```
        width = 20 ;
    }
    int rectArea ( )
    {
        return(length * width);
    }
}
class RectangleArea
{
    public static void main (string args[ ])
    Rectangle rect1 = new Rectangle ( );    // Calling constructor
    int areal = rect1.rectArea ( ) ;
    System.out.println("Area1 = "+ areal) ;
}
Output:
Area1=200;
```

The this keyword

When a method is called, it is automatically passed an implicit argument that is a reference to the invoking object (that is, the object on which the method is called). This reference is called this. To understand this, consider following example

```
class Rectangle
{
    int length, width;           // Declaration of variables
    void getData(int x, int y)    // Definition of method
    {
        this.length = x;
        this.width = y;
        int rectArea ( )    // Definition of another method
        {
            int area = this. length * this.width;
            return (area);
        }
    }
}
class RectArea                    // Class with main method
{
    public static void main (String args[ ])
    {
        int areal,area2;
        Rectangle rect1 = new Rectangle();    //Creating objects
        Rectangle rect2 = new Rectangle();
        rect1.length = 15;                    // Accessing variables
        rect1. width = 10;
        areal = rect1.length * rect1.width; I I Accessing methods
        rect2.getData(20,12);
        area2 = rect2.rectArea();
        System.out.println("Area1 = “+ areal);
```



```
        System.out.println("Area2 = "+ area2);
    }
}
```

Output:

```
Area1= 150
Area1=240
```

Overloading Methods

In Java it is possible to create methods that have the same name, but different parameter lists and different definitions. This is called method overloading. Method overloading is used when objects are required to perform similar tasks but using different input parameters. When we call a method in an object, Java matches up the method name first and then the number and type of Parameters to decide which one of the definitions to execute. This process is known as polymorphism

To create an overloaded method, all we have to do is to provide several different method definitions in the class, all with the same name, but with different parameter lists. The difference may either be in the number or type of arguments. That is, each parameter list should be unique.

Method's return type does not play any role in this. Here is an example of creating an overloaded method.

```
class Room
{
    float length;
    float breadth;
    Room(float x, float y)    // constructor1
    {
        length= x;
        breadth = y;
    }
    Room(float x)            // constructor2
    {
        length = breadth = x ;
    }
    int area ( )
    {
        return (length * breadth) ;
    }
}
```

Here, we are overloading the constructor method Room (). An object representing a rectangular room will be created as

```
Room room1 = new Room (25.0, 15.0);    //using constructor
```

On the other hand, if the room is square, then we may create the corresponding object as

```
Room room2 = new Room (20.0);    // using constructor2
```

Using Objects as Parameters

In Java it is both correct and common to pass objects to methods. Consider following example

```
class Rectangle
{
    int length;
    int width;
    Rectangle (int x, int y) // Constructor method
    {
        length = x ;
        width = y ;
    }
    void area (Rectangle r1 )
    {
        int arearect = r1.length * r1.width;
        System.out.println ("Areaa of Rectangle : " + arearect);
    }
}
class RectangleArea
{
    public static void main (string args[ ])
    Rectangle rect1 = new Rectangle (15,10);    // Calling constructor
    Rectangle rect2 = new Rectangle (20,10);
    rect2.rectArea(rect1 ) ;
}
```

Output:

Area1=150;

When you pass an object to a method, objects are implicitly passed by reference. Keep in mind that when you create a variable of a class type, you are creating a reference to an object. It is the reference, not the object itself, which is actually passed to the method. As a result, when you pass this reference to a method, the parameter that receives it will refer to the same object as that referred to by the argument. This effectively means that objects are passed to methods by use of call-by-reference.

Returning Objects

A method can return any type of data, including class types. The class `ErrorMsg` could be used to report errors. Its method, `getErrorMsg( )`, returns a `String` object that contains a description of an error based upon the error code that it is passed.

// Return a String object.

```
class ErrorMsg {
    string msgs[ ]= {"Output Error", "Input Error", "Disk Full", "Index Out-Of-Bounds"};
// Return the error message.
String getErrorMsg ( int i) {
    if (i > 0 & i < msgs.length)
        return msgs[ i];
    else
        return "Invalid Error Code:"
    }
}
```

```
class ErrMsg {  
    public static void main (String args[ ])  
        ErrMsg err = new ErrMsg();  
        System.out.println (err.getErrorMsg(2));  
        System.out.println (err.getErrorMsg(4));  
    }
```

Output:

Disk Full

Invalid Error Code

When an object is returned by a method, it remains in existence until there are no more references to it. At that point it is subject to garbage collection. Thus, an object won't be destroyed just because the method that created it terminates.

Recursion

In Java, a method can call itself. This process is called recursion, and a method that calls itself is said to be recursive. In general, recursion is the process of defining something in terms of itself and is somewhat similar to a circular definition. The key component of a recursive method is a statement that executes a call to itself. Recursion is a powerful control mechanism.

The classic example of recursion is the computation of the factorial of a number. The factorial of a number N is the product of all the whole numbers between 1 and N. For example, 3 factorial is $1 \times 2 \times 3$, or 6. The following program shows a recursive way to compute the factorial of a number. For comparison purposes, a non recursive equivalent is also included.

```
// A simple example of recursion.  
class Factorial {  
    //This is a recursive function.  
    int factR (int n)  
        int result;  
        if (n== 1 ) return 1;  
    result = factR (n-1) * n;  
    return result;  
}  
  
//This is an iterative equivalent.  
int factI ( int n) {  
    int t, result;  
    result = 1;  
    for (t=1; t<=n; t++)    result *= t;  
    return result;  
}
```

```
class Recursion {  
    public static void main (String args [ ]) {  
        Factorial f = new Factorial ();  
        System.out.println("Factorial using recursive method. ");  
    }
```

```
        System.out.println("Factorial of 3 is " + f.factR (3) );
        System.out.println("Factorial of 4 is " + f.factR (4) );
        System.out.println("Factorial of 5 is " + f.factR (5) );
        System.out.println( );
        System.out.println("Factorial using iterative method. ");
        System.out.println("Factorial of 3 is " + f.factI (3) );
        System.out.println("Factorial of 4 is " + f.factI (4) );
        System.out.println("Factorial of 5 is " + f.factI (5) );
    }
}
```

The operation of the non recursive method factI() should be clear. It uses a loop starting at 1 and progressively multiplies each number by the moving product.

The operation of the recursive factR() is a bit more complex. When factR() is called with an argument of 1, the method returns 1; otherwise, it returns the product of factR(n-1)*n. To evaluate this expression, factR() is called with n-1. This process repeats until n equals 1 and the calls to the method begin returning. For example, when the factorial of 2 is calculated, the first call to factR() will cause a second call to be made with an argument of 1. This call will return 1, which is then multiplied by 2 (the original value of n). The answer is then 2.

When a method calls itself, new local variables and parameters are allocated storage on the stack, and the method code is executed with these new variables from the start. A recursive call does not make a new copy of the method. Only the arguments are new. As each recursive call returns, the old local variables and parameters are removed from the stack, and execution resumes at the point of the call inside the method.

Understanding 'static

There will be times when you will want to define a class member that will be used independently of any object of that class. Normally a class member must be accessed through an object of its class, but it is possible to create a member that can be used by itself, without reference to a specific instance. To create such a member, precede its declaration with the keyword static. When a member is declared static, it can be accessed before any objects of its class are created, and without reference to any object. You can declare both methods and variables to be static. The most common example of a static member is main(). main() is declared as static because it must be called by the JVM when your program begins. Outside the class, to use a static member, you need only specify the name of its class followed by the dot operator. No object needs to be created. For example, if you want to assign the value 10 to a static variable called count that is part of the Timer class, use this line:

```
    Timer.count = 10;
```

This format is similar to that used to access normal instance variables through an object, except that the class name is used. A static method can be called in the same way by use of the dot operator on the name of the class.

Variables declared as static are, essentially, global variables. When an object is declared, no copy of a static variable is made. Instead, all instances of the class share the same static variable. Here is an example that shows the differences between a static variable and an instance variable:

```
//Use a static variable.
class StaticDecmo
{
```

```
int x; // a normal instance variable
static int y; // a static variable
//Return the sum of the instance variable x
// and the static variable y.
int sum ( ) {
    return x + y;
}
}
class Sdemo {
    public static void main (String args[ ] ) {
        StaticDemo ob1 = new StaticDemo ( );
        StaticDemo ob2 = new StaticDemo ( );
        // Each object has its own copy of an instance variable.
        ob1.x = 10;
        ob2.x = 20;
        System.out.println( "Of course, ob1.x and ob2.x " + "are independent. ");
        System.out.println( "ob1.x: " + ob1.x + "\nob2.x: " + ob2.x);
        System.out.println( );
        // each object share one copy of a static variable.
        System.out.println( "The static variable y is shared. ");
        StaticDemo.y = 19;
        System.out.println( "Set StaticDemo.y to 19. ");
        System.out.println( "ob1.sum( ) : " + ob1.sum( ));
        System.out.println( "ob2.sum( ) : " + ob2.sum( ));
        System.out.println( );
        StaticDemo.y = 100;
        System.out.println( "Change StaticDemo.y to 100 ");
        System.out.println( "ob1.sum( ) : " + ob1.sum( ));
        System.out.println( "ob2.sum( ) : " + ob2.sum( ));
        System.out.println( );
    }
}
```

The output of the program is shown here

Of course, ob1.x and ob2.x " + "are independent.

ob1.x =10;

ob2.x = 20;

The static variable y is shared.

Set staticDemo.y to 19.

ob1.sum() : 29

ob2.sum () : 39

change StaticDemo.y to 100

ob1.sum() : 110

ob2.sum () : 120

As you can see, the static variable y is shared by both ob1 and ob2. Changing it affects the entire class, not just an instance. The difference between a static method and a normal method is that the static method is called through its class name, without any object of that class being created. Program given below Defining and using static members class Math operation

```
class Mathoperation
{
    static float mul(float x, float y)
    {
        return x*y;
    }
    static float divide(float x, float y)
    {
        return x/y ;
    }
}
class MathApplication
{
    public void static main(string args[ ])
    {
        float a = MathOperation.mul(4.0,5.0);
        float b = MathOperation.divide(a,2.0);
        System.out.println("b = "+ b) ;
    }
}
```

Output of Above Program is:

b = 10.0

Introducing ‘final ‘

The **final keyword** in java is used to restrict the user. The java final keyword can be used in many context. Final can be:

1. variable
2. method
3. class

The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable. It can be initialized in the constructor only. The blank final variable can be static also which will be initialized in the static block only. We will have detailed learning of these. Let's first learn the basics of final keyword.

Java final variable

If you make any variable as final, you cannot change the value of final variable(It will be constant).

Example

There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

```
class Bike9{

    final int speedlimit=90;//final variable

    void run(){

        speedlimit=400;

    }

    public static void main(String args[]){

        Bike9 obj=new Bike9();

        obj.run();

    }

} //end of class
```

Java final method

if you make any method as final, you cannot override it.

```
class Bike{

    final void run(){System.out.println("running");}

}

class Honda extends Bike{

    void run(){System.out.println("running safely with 100kmph");}

    public static void main(String args[]){

        Honda honda= new Honda();

        honda.run();

    }

}
```

Java final class

If you make any class as final, you cannot extend it.

```
final class Bike{}
```

```
class Honda1 extends Bike{
```

```
void run(){System.out.println("running safely with 100kmph");}
```

```
public static void main(String args[]){
```

```
    Honda1 honda= new Honda();
```

```
    honda.run();
```

```
}
```

```
}
```

Using Command-Line Arguments

The java command-line argument is an argument i.e. passed at the time of running the java program.

The arguments passed from the console can be received in the java program and it can be used as an input.

So, it provides a convenient way to check the behavior of the program for the different values. You can pass N (1,2,3 and so on) numbers of arguments from the command prompt.

Simple example of command-line argument in java

```
class CommandLineExample{
public static void main(String args[]){
System.out.println("Your first argument is: "+args[0]);
}
}
```

```
compile by > javac CommandLineExample.java
run by > java CommandLineExample sonoo
```


Varargs : Variable-Length Arguments

Sometimes you will want to create a method that takes a variable number of arguments, based on its precise usage. For example, a method that opens an Internet connection might take a user name, password, file name, protocol, and so on, but supply defaults if some of this information is not provided. In this situation, it would be convenient to pass only the arguments to which the defaults did not apply. To create such a method implies that there must be some way to create a list of arguments that is variable in length, rather than fixed.

A variable-length argument is specified by three periods (...). For example, here is how to write a method called `vaTest()` that takes a variable number of arguments:

```
class VariableTest
{
// vaTest ( ) uses a vararg.
static void vaTest (int ... v) {
    System.out.println("Number of args: " + v.length);
    System.out.println("Contents: ");
    for (int i=0; i < v.length; i++)
        System.out.println ("arg " + i + ": " + v[i]);
    System.out.println();
}
public static void main (String args [ ] ) {
    // Notice how vaTest ( ) can be called with a
    // variable number of arguments.
    vaTest (10);          // 1 arg
    vaTest (1, 2, 3);      // 3 args
    vaTest ();             // no args
}
}
```

The output of the program is shown here:

```
Number of args: 1
Contents :
  arg 0: 10
Number of args : 3
Contents :
  args 0: 1
  args 1: 2
  args 2: 3
Number of args: 0
Contents :
```

Notice that `v` is declared as shown here:

```
int ... v
```

This syntax tells the compiler that `vaTest()` can be called with zero or more arguments. Furthermore, it causes `v` to be implicitly declared as an array of type `int []`. Thus, inside `vaTest()`, `v` is accessed using the normal array syntax.

There are two important things to notice about this program. First, as explained, inside `vaTest()`, `v` is operated on as an array. This is because `v` is an array. The ... syntax simply tells the compiler that a variable number of arguments will be used, and that these arguments will be stored

in the array referred to by v. Second, in main(), vaTest() is called with different numbers of arguments, including no arguments at all. The arguments are automatically put in an array and passed to v. In the case of no arguments, the length of the array is zero. A method can have “normal” parameters along with a variable-length parameter. However, the variable-length parameter must be the last parameter declared by the method.

Chapter 3 -Arrays and Strings

One-Dimensional Arrays

An array is a group of contiguous or related data items that share a common name. For instance, we can define an array name salary to represent a set of salaries of a group of employees.

A list of items can be given one variable name using only one subscript and such a variable is called a *single-subscripted* variable or a *one-dimensional* array. In mathematics, we often deal with variables that are single-subscripted.

In Java, single-subscripted variable x can be expressed as

x[1], x[2], x[3] x[n] The subscript can begin with number 0. That is x[0] is allowed. For example, if we want to represent a set of five numbers, say (35, 40, 20, 57, 19), by an array variable number, then we may create the variable number as follows
int number[] = **new** int[5]; and the computer reserves five storage location

The values to the array elements can be assigned as follows:

```
number[0]=35;  
number[1]=40;  
number[2]=20;  
number[3]=57;  
number[4]=19;
```

These elements may be used in programs just like any other Java variable. For example, the following are valid statements:

```
aNumber    = number [0] +10;  
number [4] = number [0] + number [2];  
number [2] = x [5] + y[10];  
value [6]   = number [i] * 3;
```

The subscript of an array can be integer constants, integer variables like i , or expressions that yield integers.

Creating an Array

Like any other variables, arrays must be declared and created in the computer memory before they are used. Creation of an array involves three steps

1. Declaring the array
2. Creating memory locations
3. Putting values into the memory locations.

Declaration of Arrays

Arrays in Java may be declared in two forms:

Form 1

```
type arrayname[ ]:
```

Form 2

```
type [ ] arrayname:
```

Examples:

```
int      number[ ];  
float    average[ ];  
int[ ]   counter;  
float[ ] marks;
```

Remember, we do not enter the size of the arrays in the declaration.

Creation of Arrays

After declaring an array, we need to create it in the memory. Java allows us to create arrays using new operator only, as shown below:

```
arrayname = new type[size];
```

Examples:

```
number = new int[5];  
average = new float[10];
```

These lines create necessary memory locations for the arrays number and average and designate them as int and float respectively. Now, the variable number refers to an array of 5 integers and average refers to an array of 10 floating point values.

It is also possible to combine the two steps declaration and creation into one as shown below:

```
int number[ ] = new int[5];
```

Figure 9.1 illustrates creation of an array in memory.

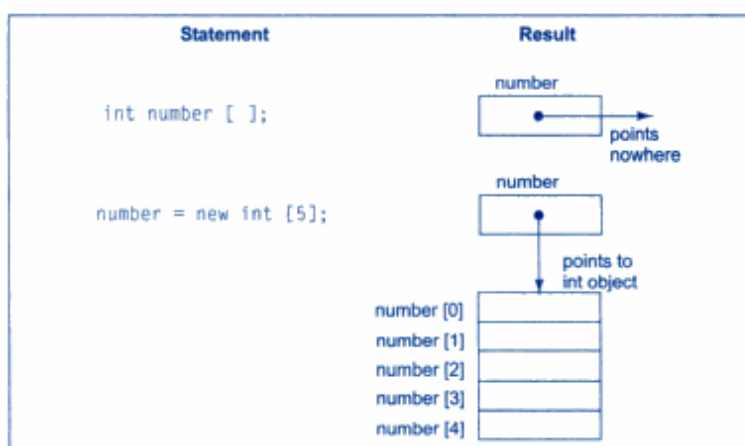


Fig. 9.1 Creation of an array in memory

Initialization of Arrays

The final step is to put values into the array created. This process is known as initialization. This is done using the array subscripts as shown below.

```
arrayname[subscript] = value ;
```

Example:

```
number[0] = 35;
number[1] = 40;
.....
number[4] = 19;
```

Note that Java creates arrays starting with the subscript of 0 and ends with a value one less than the size specified.

Unlike C, Java protects arrays from overruns and under runs. Trying to access an array bound its boundaries will generate an error message.

We can also initialize arrays automatically in the same way as the ordinary variables when they are declared, as shown below:

```
type arrayname[ ] = {list of values};
```

The array initializer is a list of values separated by commas and surrounded by curly braces. Note that no size is given. The compiler allocates enough space for all the elements specified in the list.

Example:

```
int number[] = (35, 40, 20, 57, 19);
```

It is possible to assign an array object to another. Example:

```
int a[] = (1, 2, 3);
```

```
int b[ ];
```

```
b = a;
```

are valid in Java. Both the arrays will have the same values. Loops may be used to initialize large size arrays. Example:

```
.....
for(int i = 0; i < 100; i++)
{
    if(i < 50)
        sum[i] = 0.0;
    else
        sum[i] = 1.0;
}
.....
```

The first fifty elements of the array sum are initialized to zero while the remaining are initialized to 1.0.

Consider another example as shown below:

```
for(int x=0; x < 10; x++)
```

```
average[x]= (float)x;
```

This loop initializes the array average to the values 0.0 to 9.0.

Array Length

In Java, all arrays store the allocated size in a variable named length. We can obtain the length of the array a using a.length. Example:

```
int aSize =a. length;
```

This information will be useful in the manipulation of arrays when their sizes are not known, Program 9.1 illustrates the use of an array for sorting a list of numbers.

Program 9.1 Sorting a list of numbers

```
class NumberSorting
{
    public static void main(String args[ ])
    {
        int number[] = { 55, 40, 80, 65, 71 };
        int n = number.length;           // Array length
        System.out.print("Given list : ");
        for (int i= 0; i < n; i++)
```

(Continued)

Program 9.1 (Continued)

```
{
    System.out.print(" " + number[i]);
}
System.out.println("/n");
// Sorting begins
for (int i = 0; i < n; i++)
{
    for (int j = i+1; j < n; j++)
    {
        if (number[i] < number[j])
        {
            // Interchange values
            int temp = number[i];
            number[i] = number[j];
            number[j] = temp;
        }
    }
}
// Sorting ends
System.out.print("Sorted list : ");
for (int i = 0; i < n; i++)
{
    System.out.print(" " + number[i]);
}
System.out.println(" ");
}
```

Program 9.1 displays the following output:

Given list	:	55	40	80	65	71
Sorted list	:	40	55	65	71	80

Two-dimensional Arrays

So far we have discussed the array variables that can store a list of values. There will be situations where a table of values will have to be stored. Consider the following data table, which shows the value of sales of three items by four salesgirls:

	Item1	Item2	Item3
Salesgirl #1	310	275	365
Salesgirl #2	210	190	325
Salesgirl #3	405	235	240
Salesgirl #4	260	300	380

The table contains a total of 12 values, three in each line. We can think of this table as a matrix consisting of four rows and three columns. Each row represents the values of sales by a particular salesgirl and each column represents the values of sales of a particular item.

In mathematics, we represent a particular value in a matrix by using two subscripts such as v_{ij} . Here v denotes the entire matrix and v_{ij} refers to the value in the i th row and j th column. For example, in the above table v_{22} refers to the value 325. Java allows us to define such tables of items by using two dimensional arrays. The table discussed above can be represented in Java as

`v[4][3]`

Two dimensional arrays are stored in memory as shown in Fig. 9.2. As with the single dimensional arrays, each dimension of the array is indexed from zero to its maximum size minus one; the first index selects the row and the second index selects the column within that row.

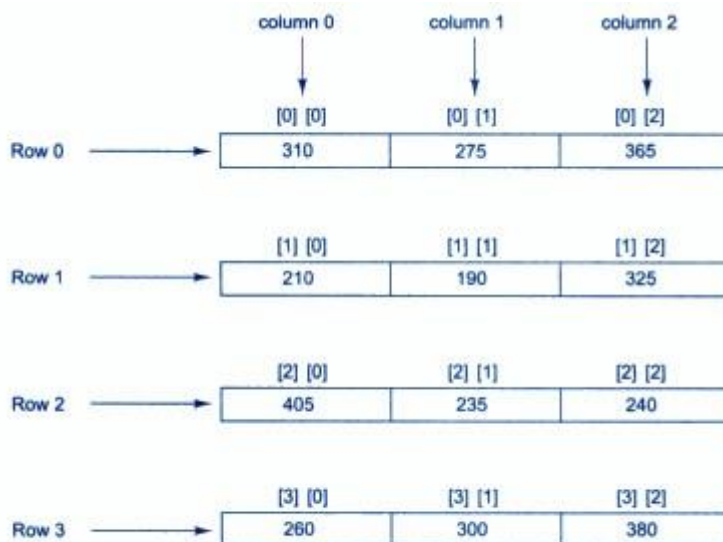


Fig. 9.2 Representation of a two-dimensional array in memory

For creating two-dimensional arrays, we must follow the same steps as that of simple arrays. We may create a two-dimensional array like this:

```
int myArray[ ][ ];
```

```
myArray = new int[3][4];
```

```
int myArray[ ][ ]= new int[3][4];
```

This creates a table that can store 12 integer values, four across and three down.

Like the one-dimensional arrays, two-dimensional arrays may be initialized by following their declaration with a list of initial values enclosed in braces. For example,

```
int table[2][3] = {0, 0, 0, 1, 1, 1};
```

initializes the elements of the first row to zero and the second row to one. The initialization is done row by row. The above statement can be equivalently written as

```
int table[ ][ ] = {{0, 0, 0}, {1, 1, 1}};
```

by surrounding the elements of each row by braces. We can also initialize a two-dimensional array in the form of a matrix as shown below:

```
int table[][] = {  
    {0, 0, 0},  
    {1, 1, 1}  
};
```

Note the syntax of the above statements. Commas are required after each brace that closes off a row, except in the case of the last row.

We can refer to a value stored in a two-dimensional array by using subscripts for both the column and row of the corresponding element. Example:

```
int value table[1][2];
```

This retrieves the value stored in the second row and third column of table matrix. A quick way to initialize a two-dimensional array is to use nested for loops as shown below:


```
for (i = 0; i < 5; i++)
{
    for (j = 0; j < 5; j++)
    {
        if (i == j)
            table[i][j] = 1;
        else
            table[i][j] = 0;
    }
}
```

This will set all the diagonal elements to 1 and others to zero as given below :

1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

Program 9.2 illustrates the use of two-dimensional arrays in real-life situations.

Program 9.2 Application of two-dimensional arrays

```
class MulTable
{
    final static int ROWS = 20;
    final static int COLUMNS = 20;
    public static void main(String args[ ])
    {
        int product[][] = new int[ROWS][COLUMNS];
        int row, column;
        System.out.println("MULTIPLICATION TABLE");
        System.out.println(" ");
    }
}
```

(Continued)

Program 9.2 (Continued)

```
int i,j;
for (i=10; i<ROWS; i++)
{
    for (j=10; j<COLUMNS; j++)
    {
        product[i][j] = i*j;
        System.out.print(" " +product[i][j]);
    }
    System.out.println(" ");
}
}
```

Program 9.2 produces the following output:

MULTIPLICATION TABLE									
100	110	120	130	140	150	160	170	180	190
110	121	132	143	154	165	176	187	198	209
120	132	144	156	168	180	192	204	216	228
130	143	156	169	182	195	208	221	234	247
140	154	168	182	196	210	224	238	252	266
150	165	180	195	210	225	240	255	270	285
160	176	192	208	224	240	256	272	288	304
170	187	204	221	238	255	272	289	306	323
180	198	216	234	252	270	288	306	324	342
190	209	228	247	266	285	304	323	342	361

Variable Size Arrays

Java treats multidimensional array as "arrays of arrays". It is possible to declare a two-dimensional array as follows:

```
int x[ ][ ] = new int[3][ ];
```

```
x[0] = new int[2];
```

```
x[1] = new int[4];
```

```
x[2] = new int[3];
```

These statements create a two-dimensional array as having different lengths for each row as shown in Fig. 9.3.

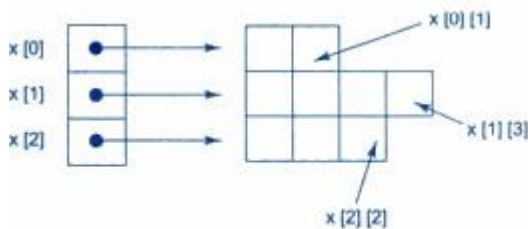


Fig. 9.3 Variable size arrays

Strings

String manipulation is the most common part of many Java programs. Strings represent a sequence of characters. The easier way to represent a sequence of characters in Java is by using a character array. Example:

```
char charArray[ ] = new char[4];
```

```
charArray[0] = "J" ;
```

```
charArray[1] ="a";
```

```
charArray[2] = "v";
```

```
charArray[3] = "a";
```

Although character arrays have the advantage of being able to query their length, they themselves are not good enough to support the range of operations we may like to perform on strings. For example, copying one character array into another might require a lot of book keeping effort. Fortunately, Java is equipped to handle these situations more efficiently.

In Java, strings are class objects and implemented using two classes, namely, String and StringBuffer. A Java string is an instantiated object of the String class. Java strings, as compared to C strings, are more reliable and predictable. This is basically due to C's lack of bounds-

checking. A Java string is not a character array and is not NULL terminated. Strings may be declared and created as follows:

```
String stringName;  
StringName = new String ("string");
```

Example:

```
String firstName;  
  
firstName = new String("Anil");
```

These two statements may be combined as follows:

```
String firstName = new String ("Anil");
```

Like arrays, it is possible to get the length of string using the length method of the String class.

```
int m = firstName.length();
```

Note the use of parentheses here. Java strings can be concatenated using the + operator.

Examples:

```
String fullName = name1 + name2;
```

```
String city1 = "New" + "Delhi";
```

where name1 and name2 are Java strings containing string constants. Another example is:

```
System.out.println(firstName + "Kumar");
```

String Arrays

We can also create and use arrays that contain strings. The statement

```
String itemArray[ ] = new String[3];
```

will create an itemArray of size 3 to hold three string constants. We can assign the strings to the itemArray element by element using three different statements or more efficiently using a for loop.

String Methods

The String class defines a number of methods that allow us to accomplish a variety of string manipulation tasks. Table 9.1 lists some of the most commonly used string methods, and their tasks. Program 9.3 shows the use of the method compareTo() to sort an array of strings in alphabetical order.

Table 9.1 Some Most Commonly Used String Methods

Method Call	Task performed
<code>s2 = s1.toLowerCase();</code>	Converts the string s1 to all lowercase
<code>s2 = s1.toUpperCase();</code>	Converts the string s1 to all Uppercase
<code>s2 = s1.replace('x', 'y');</code>	Replace all appearances of x with y
<code>s2 = s1.trim();</code>	Remove white spaces at the beginning and end of the string s1
<code>s1.equals(s2)</code>	Returns 'true' if s1 is equal to s2
<code>s1.equalsIgnoreCase(s2)</code>	Returns 'true' if s1 = s2, ignoring the case of characters
<code>s1.length()</code>	Gives the length of s1
<code>s1.charAt(n)</code>	Gives nth character of s1
<code>s1.compareTo(s2)</code>	Returns negative if s1 < s2, positive if s1 > s2, and zero if s1 is equal s2
<code>s1.concat(s2)</code>	Concatenates s1 and s2
<code>s1.substring(n)</code>	Gives substring starting from n th character
<code>s1.substring(n, m)</code>	Gives substring starting from n th character up to m th (not including m th)
<code>String.valueOf(p)</code>	Creates a string object of the parameter p (simple type or object)
<code>p.toString()</code>	Creates a string representation of the object p
<code>s1.indexOf('x')</code>	Gives the position of the first occurrence of 'x' in the string s1
<code>s1.indexOf('x', n)</code>	Gives the position of 'x' that occurs after nth position in the string s1
<code>String.valueOf(Variable)</code>	Converts the parameter value to string representation

StringBuffer Class

StringBuffer is a peer class of String. While String creates strings of fixed length, StringBuffer creates strings of flexible length that can be modified in terms of both length and content. We can insert characters and substrings in the middle of a string, or append another string to the end. Table 9.2 lists some of the methods that are frequently used in string manipulations.

Table 9.2 Commonly Used StringBuffer Methods

Method	Task
<code>s1.setCharAt(n, 'x')</code>	Modifies the nth character to x
<code>s1.append(s2)</code>	Appends the string s2 to s1 at the end
<code>s1.insert(n, s2)</code>	Inserts the string s2 at the position n of the string s1
<code>s1.setLength(n)</code>	Sets the length of the string s1 to n. If $n < s1.length()$ s1 is truncated. If $n > s1.length()$ zeros are added to s1

Vectors

We have seen that J2SE 5.0 version supports the concept of variable arguments to methods. This feature can also be achieved in Java through the use of the Vector class contained in the java.util package. This class can be used to create a generic dynamic array known as vector that can hold objects of any type and any number. The objects do not have to be homogeneous. Arrays can be easily implemented as vectors. Vectors are created like arrays as follows:

```
Vector intVect = new Vector(); // declaring without size
```

```
Vector list = new Vector (3); // declaring with size
```

Note that a vector can be declared without specifying any size explicitly. A vector without size can accommodate an unknown number of items. Even, when a size is specified, this can be overlooked and a different number of items may be put into the vector. Remember, in contrast, an array must always have its size specified.

Vectors possess a number of advantages over arrays.

1. It is convenient to use vectors to store objects.
2. A vector can be used to store a list of objects that may vary in size.
3. We can add and delete objects from the list as and when required.

A major constraint in using vectors is that we cannot directly store simple data type in a vector, we can only store objects. Therefore, we need to convert simple types to objects. This can be done using the wrapper classes discussed in the next section. The vector class supports a number of methods that can be used to manipulate the vectors created. Important ones are listed in Table 9.3.

Table 9.3 Important Vector Methods

Method Call	Task performed
<code>list.addElement(item)</code>	Adds the item specified to the list at the end
<code>list.elementAt(10)</code>	Gives the name of the 10th object
<code>list.size()</code>	Gives the number of objects present
<code>list.removeElement(item)</code>	Removes the specified item from the list
<code>list.removeElementAt(n)</code>	Removes the item stored in the nth position of the list
<code>list.removeAllElements()</code>	Removes all the elements in the list
<code>list.copyInto(array)</code>	Copies all items from list to array
<code>list.insertElementAt (item, n)</code>	Inserts the item at nth position

Program 9.5 illustrates the use of arrays, strings and vectors. This program converts a string vector into an array of strings and displays the strings.

Program 9.5 Working with vectors and arrays

```
import java.util.*;           // Importing Vector class
class LanguageVector
{
    public static void main(String args[ ])
    {
```

(Continued)

Program 9.5 (Continued)

```
        Vector list = new Vector();
        int length = args.length;
        for (int i = 0; i < length; i++)
        {
            list.addElement(args[i]);
        }
        list.insertElementAt("COBOL",2);
        int size = list.size();
        String listArray[] = new String[size];
        list.copyInto(listArray);
        System.out.println("List of Languages");
        for (int i = 0; i < size; i++)
        {
            System.out.println(listArray[i]);
        }
    }
}
```

Command line input and output are:

```
C:\JAVA\prog>java LanguageVector Ada BASIC C++ FORTRAN Java
List of Languages
Ada
BASIC
COBOL
C++
FORTRAN
Java
```


Wrapper Classes

As pointed out earlier, vectors cannot handle primitive data types like int, float, long, char, and double. Primitive data types may be converted into object types by using the wrapper classes contained in the java.lang package. Table 9.4 shows the simple data types and their corresponding wrapper class types.

Table 9.4 Wrapper Classes for Converting Simple Types	
<i>Simple Type</i>	<i>Wrapper Class</i>
boolean	Boolean
char	Character
double	Double
float	Float
int	Integer
long	Long

The wrapper classes have a number of unique methods for handling primitive data types and objects. They are listed in the following tables.

Table 9.5 Converting Primitive Numbers to Object Numbers Using Constructor Methods	
<i>Constructor Calling</i>	<i>Conversion Action</i>
Integer IntVal = new Integer(i);	Primitive integer to Integer object
Float FloatVal = new Float(f);	Primitive float to Float object
Double DoubleVal = new Double(d);	Primitive double to Double object
Long LongVal = new Long(l);	Primitive long to Long object

Note: *i, f, d and l are primitive data values denoting int, float, double and long data types. They may be constants or variables.*

Table 9.6 Converting Object Numbers to Primitive Numbers Using typeValue() method	
<i>Method Calling</i>	<i>Conversion Action</i>
int i = IntVal.intValue();	Object to primitive integer
float f = FloatVal.floatValue();	Object to primitive float
long l = LongVal.longValue();	Object to primitive long
double d = DoubleVal.doubleValue();	Object to primitive double

Table 9.7 Converting Numbers to Strings Using toString() Method	
<i>Method Calling</i>	<i>Conversion Action</i>
str = Integer.toString(i)	Primitive integer to string
str = Float.toString(f);	Primitive float to string
str = Double.toString(d);	Primitive double to string
str = Long.toString(l);	Primitive long to string

**Table 9.8 Converting String Objects to Numeric Objects
Using the Static Method ValueOf()**

<i>Method Calling</i>	<i>Conversion Action</i>
<code>DoubleVal = Double.valueOf(str);</code>	Converts string to Double object
<code>FloatVal = Float.valueOf(str);</code>	Converts string to Float object
<code>IntVal = Integer.valueOf(str);</code>	Converts string to Integer object
<code>LongVal = Long.valueOf(str);</code>	Converts string to Long object

Note: These numeric objects may be converted to primitive numbers using the `intValue( )` method as shown in Table 9.6.

**Table 9.9 Converting Numeric Strings to Primitive Numbers
Using Parsing Methods**

<i>Method Calling</i>	<i>Conversion Action</i>
<code>int i = Integer.parseInt(str);</code>	Converts string to primitive integer
<code>long i = Long.parseLong(str);</code>	Converts string to primitive long

Note: `parseInt( )` and `parseLong( )` methods throw a `NumberFormatException` if the value of the `str` does not represent an integer.